trajeR package

Cédric NOEL - Jang SHILTZ
cedric.noel@univ-lorraine.fr

June 24, 2020

Contents

1 Introduction **3**

2 Getting started **3**

3 LOGIT model **5**
3.1 Parameters 5
3.2 Plot 9

4 Censored Normal distribution **16**

5 Censored Normal model **16**
5.1 Parameters 17
5.2 Plot 21
5.3 Adding covariate 23

6 Censored Data **27**

7 Zero Inflated Poisson **33**
7.1 Parameters 34
7.2 Plot 37
7.3 Adding covariate 39
7.4 Adding time depdendent covariate 41

8 Non Linear model **43**
8.1 Parameters 43

9 Membership probability **48**

10 Profiles of group **49**

11 Information criteria **49**

12 Model adequacy **49**

1 Introduction

Longitudinal studies are often employed on several disciplines like finance, econometrics, psychology, sociology, biology, ...

The aims of this package is to give tools to work with this situation.

More precisely, we consider a variable Y that evolves over time.

2 Getting started

First, we need to load `trajeR` package.

```
library(trajeR)
```

The main function is named `trajeR`. It fit the model and find parameters to a given degre of polynomial shape. It syntax is

```
trajeR(Y, A, Risk = NULL, TCOV = NULL, ng, degre, degre.nu = 0,
       Model, Method = "L",
       ssigma = FALSE, ymax = max(Y) + 1, ymin = min(Y) - 1,
       hessian = TRUE, itermax = 100, paraminit = NULL,
       ProbIRLS = TRUE, refgr = 1,
       fct = NULL, diffct = NULL, nbvar = NULL, nls.lmiter = 50)
```

Arguments

`Y`	a matrix containing the variables in the model.
`A`	a matrix containing the time variable data.
`Risk`	an optionnal matrix that modifie the probabilty of belong to group. By default its value is a mtrix with one column with value 1.
`TCOV`	an optionnal matrix containing the time covariate that influence the trajectory themselve. By default its value is NULL.
`ng`	integer. The number of group.
`degre`	vector of integer. The degre of every ploynomial function.
`degre.nu`	vecor of integer. The degre of all Poisson part for a ZIP model.
`Model`	the model used. The value are LOGIT for a Logit Mixture model, CNORM for a Censored Normal Mixture Model or ZIP for Zero Inflated Poisson Mixture model.
`Method`	determine the method used for find the paramters of the model. The value are L for the Maximum Likelihiood Estimation, EM for Expectation Maximization method with quasi newton method inside, EMIWRLS for Expectation Maximization method with Iterative Weighted ??
`ssigma`	logical. By default its value is FALSE. For the CNORM model, indicate if we want the same sigma for all nomrmal density function.

`ymax` real. For the CNORM model, indicate the maximum value of the data. It oncerne only the model with censored data. By default its value is the maximum value of the data plus 1.

`ymin` real. For the CNORM model, indicate the minimum value of the data. It oncerne only the model with censored data. By default its value is the maximum value of the data minus 1.

`hessian` logical. Indicate if we want calculate the hessian matrix. Default is FALSE. If the method use is Likelihood, the hessian is calculated by inversing the Information's Fisher Matrix. To avoid numerically singular matrix we find the pseudo inverse matrix by using the ginv function int he package MASS. If the method is EM or EMIWRLS, the hessian is calculted by using Louis method.

`itermax` integer. Indicate the maximal number of iteration for optim function or for the EM algorithm.

`paraminit` vector. The vector of initial parameters. By default trajeR calculate the initial value based of the range or the standrad deviation.

`ProbIRLS` logical. Indicate the method to sue in the search of predictor's probability. If TRUE (by default) we use IRLS method and if FALSE we use optimization method.

`refgr` integer. The number of reference group. By default is 1.

`fct` a function. The definition of the function f in the definition in nonlinear model.

`diffct` a function. The differential of the function f in the nonlinear model.

`nbvar` integer. The number of variable in the nonlinear model.

`nls.lmiter` integer. In the case of non linear model, the maximum number of iterations allowed.

Details

Models for trajeR is, by default, a polynomial regression of the time value parameters for each groups. The number fo group is controlled by the integer `ng`. We can spcecify the degre of the polynomial shape for each groups by the vector `degre`.

Value

return an object of class "`Trajectory.LOGIT`". The generic accessor functions `beta`, `delta`, `theta`, `sd`, `tab`, `Likelihood`, `ng`, `model` and `method` extract various useful features of the value returned by `trajeR`.

An object of class "`Trajectory.LOGIT`" is a list containing at least the following components:

`beta` a vector of the parameters beta.

`delta` a vector of the parameter delta. Only if we use time covariate.

`theta` a vector with the paramter theta if there exist a coavriate X that modifie the probability or the probability of group membership.

`sd`	a vector of the standrad deviation of the parameters.
`tab`	a matrix with all the parameters and standard deviation.
`Likelihood`	a real with the Likelihhod obtnaied by the parameters.
`ng`	a integer with the number of group.
`model`	a string with the model used.
`method`	a string with the method used.

3 LOGIT model

In this section we consider a latent variable y^*_{it} such that

$$y^*_{it} = f(a_{it}; \beta_k, \delta_k) + \epsilon_{it} = \beta_k A_{it} + \delta_k W_t + \epsilon_{it} \tag{1}$$

where $\epsilon_{it} \sim \mathcal{N}(0;\ \sigma_k)$, $A_{it} = (1, a_{it}, a^2_{it}, \cdots, a^{n_\beta - 1}_{it})^t$, $W_t = (w_{i1}, \cdots, w_{in_\delta})^t$, $\beta_k = (\beta_{k1}, \cdots, \beta_{kn_\beta})$ and $\delta_k = (\delta_{k1}, \cdots, \delta_{kn_\delta})$.
Under this affirmation, classically, it is assumed that the binary variable $y_{it} = 1$ if $y^*_{it} > 0$ and $y_{it} = 0$ if $y^*_{it} \leq 0$.

If ϵ_{it} is assumed to be normally distributed , described the probit function. Let $\rho_{ikt} = P(Y_{it} = 1 | W_i = w_i, C_i = k)$ the probability of $y_{it} = 1$ given membership in group k.

$$\rho_{ikt} = \frac{e^{\beta_k A_{it} + \delta_k W_{it}}}{1 + e^{\beta_k A_{it} + \delta_k W_{it}}} \tag{2}$$

3.1 Parameters

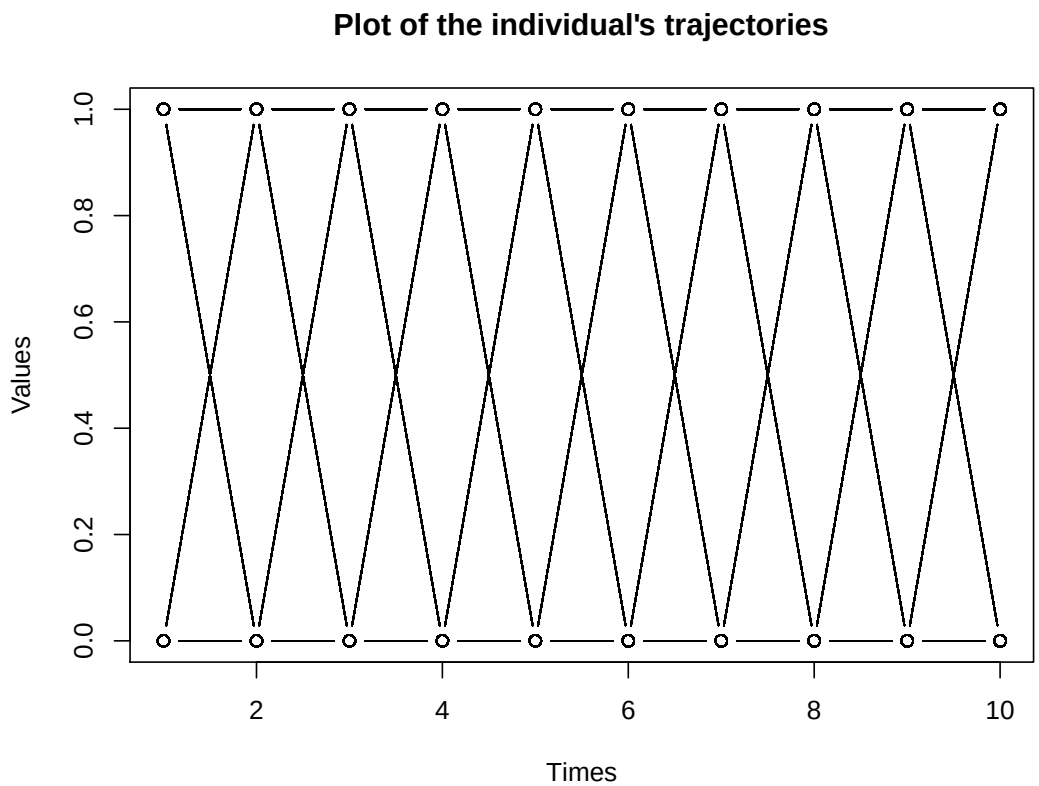
Example
We use artificial data contained in the library. This are 500 longitudinal data of 0 and 1 values generated by the parameters above :

- 3 groups ;
- probability is $\pi_1 = 0.22$, $\pi_2 = 0.44$ and $\pi_3 = 0.34$;
- period is 10 ;
- we use 3 polynomial shape
  1. degre 3 and $\beta_1 = (-2.266, -0.109, 0.14, -0.011)$;
  2. degre 4 and $\beta_2 = (1.291, 0.811, -0.554, 0.077, -0.003)$;
  3. degre 0 and $\beta_3 = (-1.99243)$.

The data contained the time variable dependent Y_i in `data[,2:11]`, the time variable in `data[,12:21]`, the time covariate in `data[,24:33]` and a covariate that influence the belonging probabilty in `data[,48:49]`.

- `data[,2:11]` is a matrix with 0 and 1 values.
- `data[,12:21]` is a matrix with time 1 to 10.

- `data[,24:33]` is a time covriate that influence the shape fo trajectory. It is a matrix with 0 and 1 value, like the presence or not of a characteristic on the individual.
- `data[,48:49]` is a matrix with 0 and 1 value.



We use the each method to fit the model. For all method, we specify the number of group of our model, ng=3, the degree of the polynomial shape of the trajectories. Here we choice a line parallel to abscissa axis, a cubic polynomial and a quadric polynomial. So degree is vector $(0, 3, 4)$.
We specify `hessian=TRUE` to ask the calculus of the hessian matrix.
`Itermax` is set to 300 to assure a good approximation of the parameters.

For the Likelihood method we call `trajeR` with option `Method ="L"`.

```
# Likelihood
solL = trajeR(Y = data[,2:11], A = data[,12:21],
              ng = 3, degre = c(0,3,4),
              Model = "LOGIT", Method = "L", hessian = TRUE)
```

```
solL

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
```

```
## Model : Logit
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -1.94507      0.10076   -19.30301          0
##
##       2   Intercept   -1.78975      0.03515   -50.91994          0
##              Linear   -0.42428          NaN         NaN        NaN
##           Quadratic    0.18356          NaN         NaN        NaN
##               Cubic   -0.01214          NaN         NaN        NaN
##
##       3   Intercept    1.73767          NaN         NaN        NaN
##              Linear    0.65316      0.07243     9.01811          0
##           Quadratic   -0.55808      0.02473   -22.56348          0
##               Cubic    0.08237      0.00195    42.26117          0
##             Quartic   -0.00337          NaN         NaN        NaN
## ---------------------------------------------------------------------
##       1         pi1     0.4182      0.10904    11.58593          0
##       2         pi2    0.21835      0.11526     5.32299          0
##       3         pi3    0.36345      0.06269    17.91401          0
## ---------------------------------------------------------------------
## Likelihood : -2852.746
```

For the EM method, we use the same parameters except Method = EM or EMIRLS. We specifiy `itermax = 300` to ensure convergence.

```
#EM
solEM = trajeR(Y = data[,2:11], A = data[,12:21],
               ng = 3, degre = c(0,3,4),
               Model = "LOGIT", Method = "EM", hessian = TRUE, itermax = 300)
#EMIRLS
solEMIRLS = trajeR(Y = data[,2:11], A = data[,12:21],
                   ng = 3, degre=c(0,3,4),
                   Model = "LOGIT", Method = "EMIRLS", hessian = TRUE,
                   itermax = 300)
```

```
solEM

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
## Method : Expectation-maximization
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -1.94767      0.09785   -19.90445          0
```

```
##
##       2   Intercept   -1.34108      0.81571    -1.64406    0.10023
##              Linear   -0.61486      0.49091    -1.25249    0.21045
##           Quadratic    0.20768      0.09134     2.27363    0.02303
##               Cubic   -0.01308      0.00517    -2.53028    0.01143
##
##       3   Intercept    1.75148      0.80595     2.17319    0.02981
##              Linear    0.79019      0.81427     0.97043    0.33188
##           Quadratic    -0.6123      0.26768     -2.2874    0.02221
##               Cubic    0.08893      0.03451     2.57699    0.00999
##             Quartic   -0.00362      0.00152    -2.38174    0.01727
## ---------------------------------------------------------------------
##       1         pi1    0.41731      0.02973    14.03492          0
##       2         pi2    0.23237      0.02801     8.29687          0
##       3         pi3    0.35033      0.04085     8.57665          0
## ---------------------------------------------------------------------
## Likelihood : -2852.915

solEMIRLS

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
## Method : Expectation-maximization with IRLS
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -1.94514      0.09673   -20.10856          0
##
##       2   Intercept    -1.7818      1.01371    -1.75769    0.07886
##              Linear     -0.426       0.5851    -0.72809    0.46659
##           Quadratic     0.1834      0.10483     1.74952    0.08026
##               Cubic   -0.01211      0.00578    -2.09538    0.03619
##
##       3   Intercept    1.74118      0.76282     2.28256     0.0225
##              Linear    0.65271      0.77462     0.84262    0.39948
##           Quadratic   -0.55852      0.25545    -2.18637    0.02883
##               Cubic    0.08247      0.03302     2.49736    0.01254
##             Quartic   -0.00338      0.00146    -2.31562    0.02062
## ---------------------------------------------------------------------
##       1         pi1    0.41814      0.02941    14.21838          0
##       2         pi2    0.21874      0.02786     7.85256          0
##       3         pi3    0.36312      0.04051     8.96429          0
## ---------------------------------------------------------------------
## Likelihood : -2852.746
```

We can add risk covariate that influence the belonging probability. By default, the effect of risk variable are compared to the first group that is reference group but we can change this setting by the option `refgr`.

```r
solLRisk = trajeR(Y = data[,2:11], A = data[,12:21], Risk = data[,22:23],
                  ng = 3, degre = c(0,3,4),
                  Model = "LOGIT", Method = "L", hessian = TRUE,
                  itermax = 300)
```

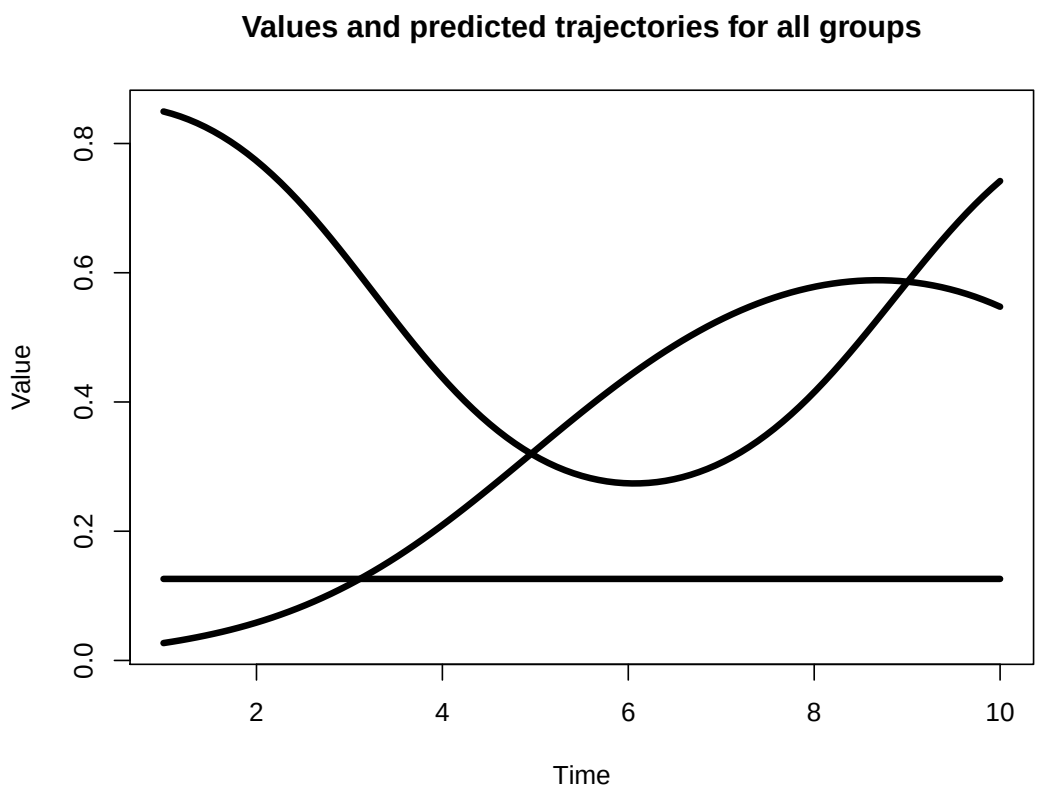
```r
solLRisk

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -1.93507      0.17823   -10.85714          0
##
##       2   Intercept   -4.43095      0.20222   -21.91122          0
##              Linear    0.85167      0.20047     4.24844      2e-05
##           Quadratic   -0.00567      0.11608    -0.04889    0.96101
##               Cubic   -0.00333      0.14301    -0.02327    0.98143
##
##       3   Intercept    1.65751      0.14319    11.57594          0
##              Linear    0.49393      0.09641     5.12329          0
##           Quadratic   -0.49172      0.06154    -7.98966          0
##               Cubic    0.07468          NaN         NaN        NaN
##             Quartic    -0.0031          NaN         NaN        NaN
## ---------------------------------------------------------------------
##       1    Baseline          0          NaN         NaN        NaN
##
##       2   Intercept   -0.99578       0.0235    11.07022          0
##                  X1   -0.07114      0.00183   566.00527          0
##                  X2    0.36742          NaN         NaN        NaN
##
##       3   Intercept    0.22793       0.1473    10.07352          0
##                  X1   -0.24249       0.1575     5.47326          0
##                  X2   -0.38484      0.15668     3.96338      7e-05
## ---------------------------------------------------------------------
## Likelihood : -2849.035
```

3.2 Plot

We plot only the trajectory on a graph. We have just to use the native plot function. It use the class of the object to draw the polynomial shape. The values in abscissa are the first row of the time variable.

```r
plot(solLRisk)
```

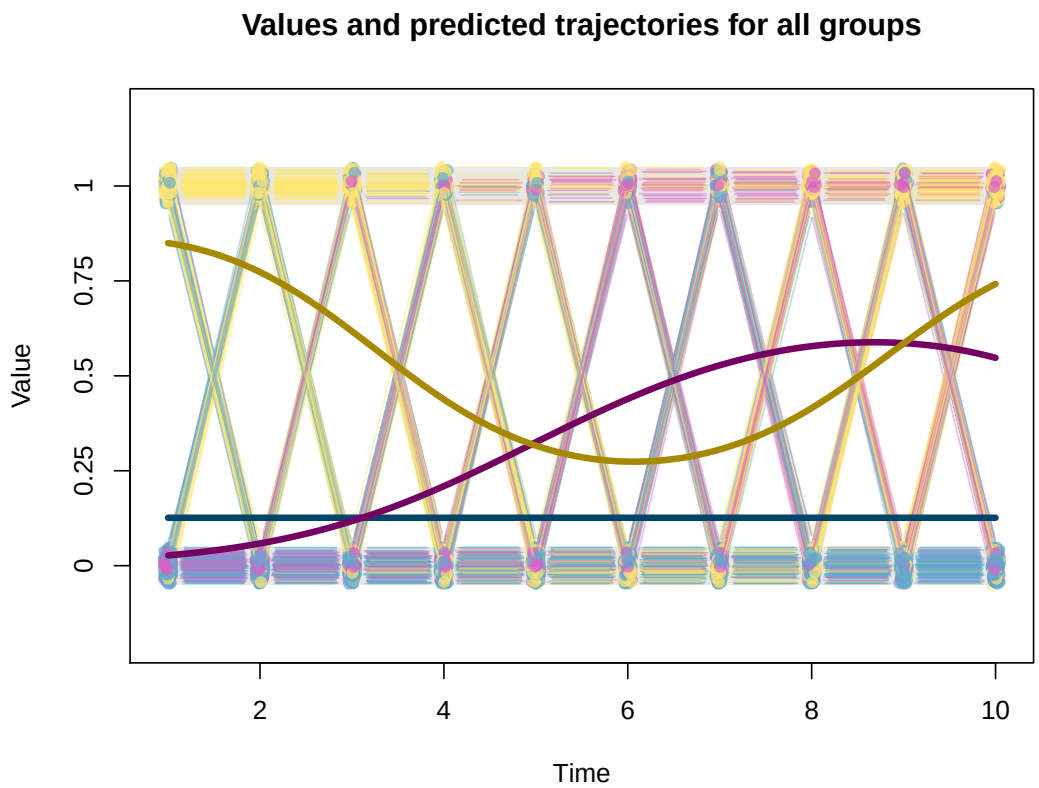


We can add longitudinal data to this graph. If we want them on the plot we have to specify Y and A in the function `plot()`.
For more visibility, we have enlarge the 0 and 1 value on the graph and plot the data with a little and random shift, control by the index `dec` in the function plot.

By default colors are gray scale, but we can specify colors we want.

```r
# colour's defintion
trans = "70"
col1 = "#034569"
col1.1 = paste0("#64AAD0", trans)
col2 = "#750062"
col2.1 = paste0("#D962C7", trans)
col3 = "#A68900"
col3.1 = paste0("#FFE773", trans)
cols1 = c(col1.1, col2.1, col3.1)
cols2 = c(col1, col2, col3)
vcol = c(cols1, cols2)
```

```r
plot(solLRisk, Y = data[,2:11], A = data[,12:21], dec = 5, col = vcol)
```

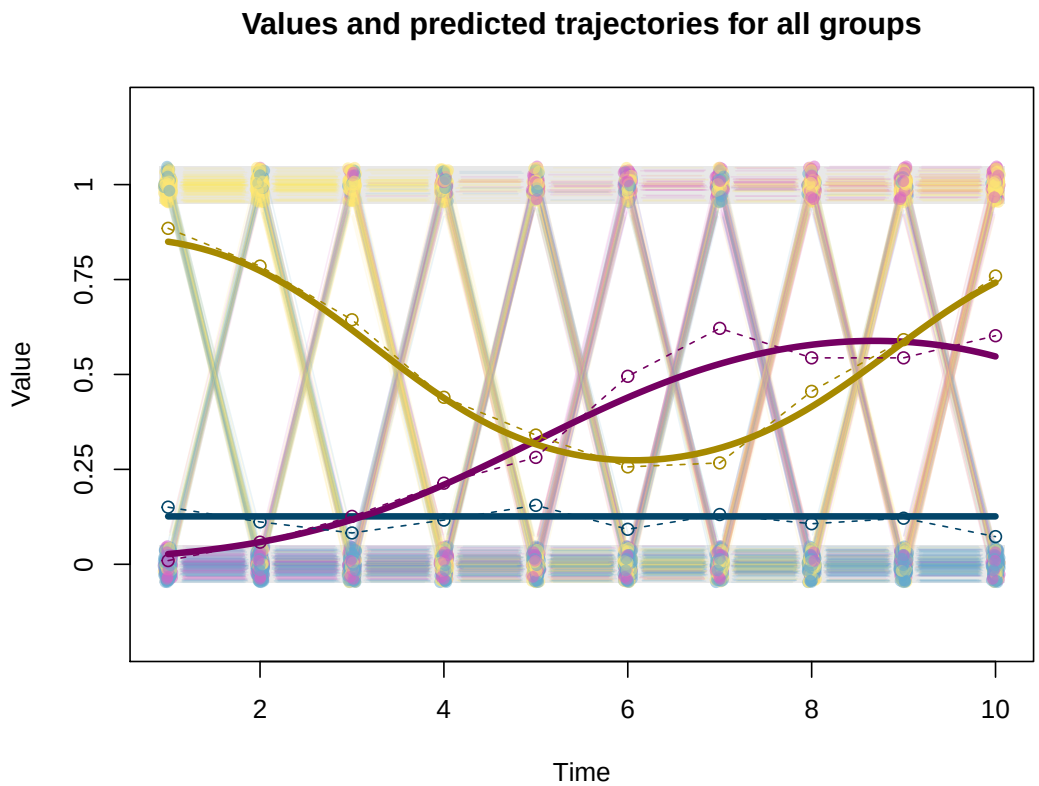


We can add the average of the data points on the plot for each time values. For each group and each time the mean of the data are calculated and add to the plot.

We use option `mean =TRUE`.

```r
# colour's defintion
trans = "70"
col1 = "#034569"
col1.1 = paste0("#64AAD0", trans)
col2 = "#750062"
col2.1 = paste0("#D962C7", trans)
col3 = "#A68900"
col3.1 = paste0("#FFE773", trans)
cols1 = c(col1.1, col2.1, col3.1)
cols2 = c(col1, col2, col3)
vcol = c(cols1, cols2)

plot(solLRisk, Y = data[,2:11], A = data[,12:21], dec = 5,
     col = vcol, mean = TRUE, alpha = 0.3)
```



Time covariate can influence directly the shape trajectories. In the data, we have time covariate that is composed by 0 or 1 value. We consider their effects by use the option `TCOV` in the command `trajeR`. We can use any of three method above.

```
solLTCOV = trajeR(Y = data[,2:11], A =data[,12:21], TCOV = data[,24:33],
                  ng = 3, degre = c(0,3,4),
                  Model = "LOGIT", Method = "L", hessian = TRUE,
                  itermax = 300)
solEMTCOV = trajeR(Y = data[,2:11], A = data[,12:21], TCOV = data[,24:33],
                   ng = 3, degre = c(0,3,4),
                   Model = "LOGIT", Method = "EM", hessian = TRUE,
                   itermax = 300)
solEMIRLSTCOV = trajeR(Y = data[,2:11], A = data[,12:21], TCOV = data[,24:33],
                       ng = 3, degre = c(0,3,4),
                       Model = "LOGIT", Method = "EMIRLS", hessian = FALSE,
                       itermax = 300)
```

```
solLTCOV

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
```

```
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -2.01722      0.14162   -14.24348          0
##               TCOV1    0.14193      0.17276     0.82159    0.41135
##
##       2   Intercept   -1.85017      0.04087   -45.27308          0
##              Linear   -0.42646          NaN         NaN        NaN
##           Quadratic    0.18422          NaN         NaN        NaN
##               Cubic   -0.01217          NaN         NaN        NaN
##               TCOV1    0.11976      0.19285     0.62102    0.53461
##
##       3   Intercept    1.72114          NaN         NaN        NaN
##              Linear    0.66158      0.08123     8.14433          0
##           Quadratic   -0.56087      0.02614   -21.45446          0
##               Cubic    0.08271      0.00202     40.8809          0
##             Quartic   -0.00339          NaN         NaN        NaN
##               TCOV1    0.01877      0.11409     0.16453    0.86932
## ---------------------------------------------------------------------
##       1         pi1    0.41872      0.10831    11.67955          0
##       2         pi2    0.21779      0.11334     5.39414          0
##       3         pi3    0.36348      0.06307    17.81401          0
## ---------------------------------------------------------------------
## Likelihood : -2851.937
```

```
solEMTCOV
```

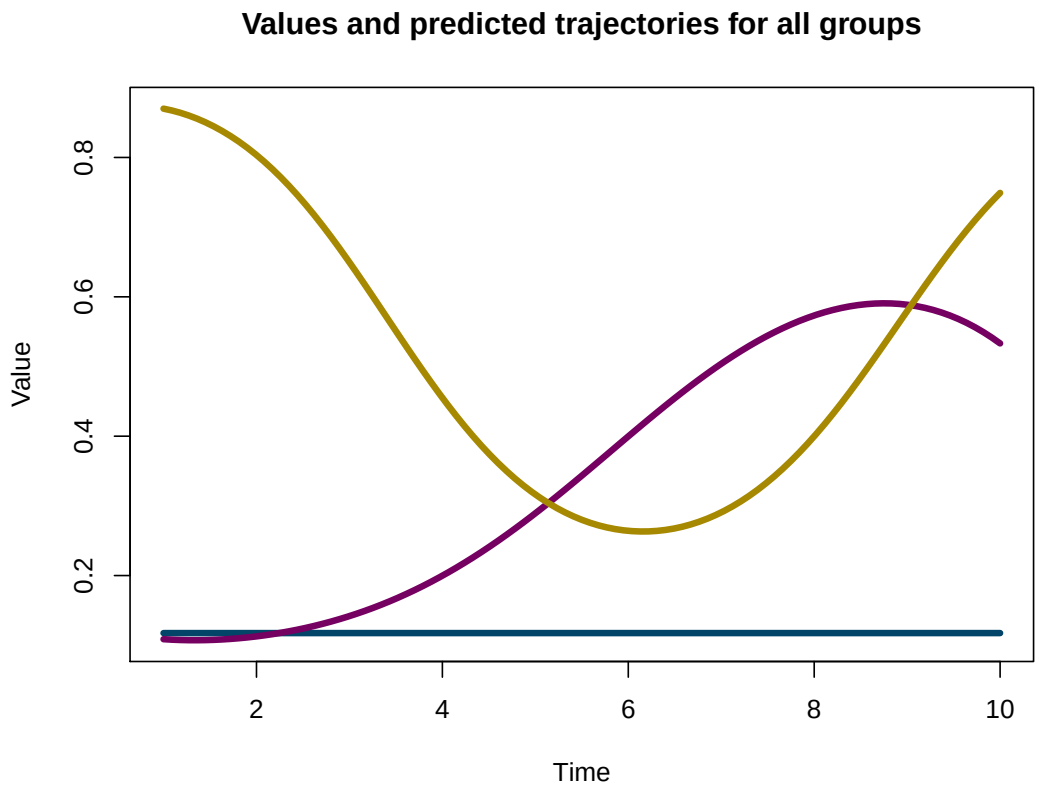
```
## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
## Method : Expectation-maximization
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -2.02089           NA          NA         NA
##               TCOV1    0.14286           NA          NA         NA
##
##       2   Intercept   -1.60807           NA          NA         NA
##              Linear   -0.53168           NA          NA         NA
##           Quadratic    0.19815           NA          NA         NA
##               Cubic   -0.01275           NA          NA         NA
##               TCOV1    0.11451           NA          NA         NA
##
##       3   Intercept    1.73803           NA          NA         NA
##              Linear    0.70029           NA          NA         NA
##           Quadratic   -0.57695           NA          NA         NA
##               Cubic    0.08464           NA          NA         NA
```

```
##             Quartic   -0.00346           NA          NA          NA
##               TCOV1    0.01458           NA          NA          NA
## ---------------------------------------------------------------------
##       1         pi1    0.41768           NA          NA          NA
##       2         pi2    0.22429           NA          NA          NA
##       3         pi3    0.35804           NA          NA          NA
## ---------------------------------------------------------------------
## Likelihood : -2851.965
```

```
solEMIWRLSTCOV
```

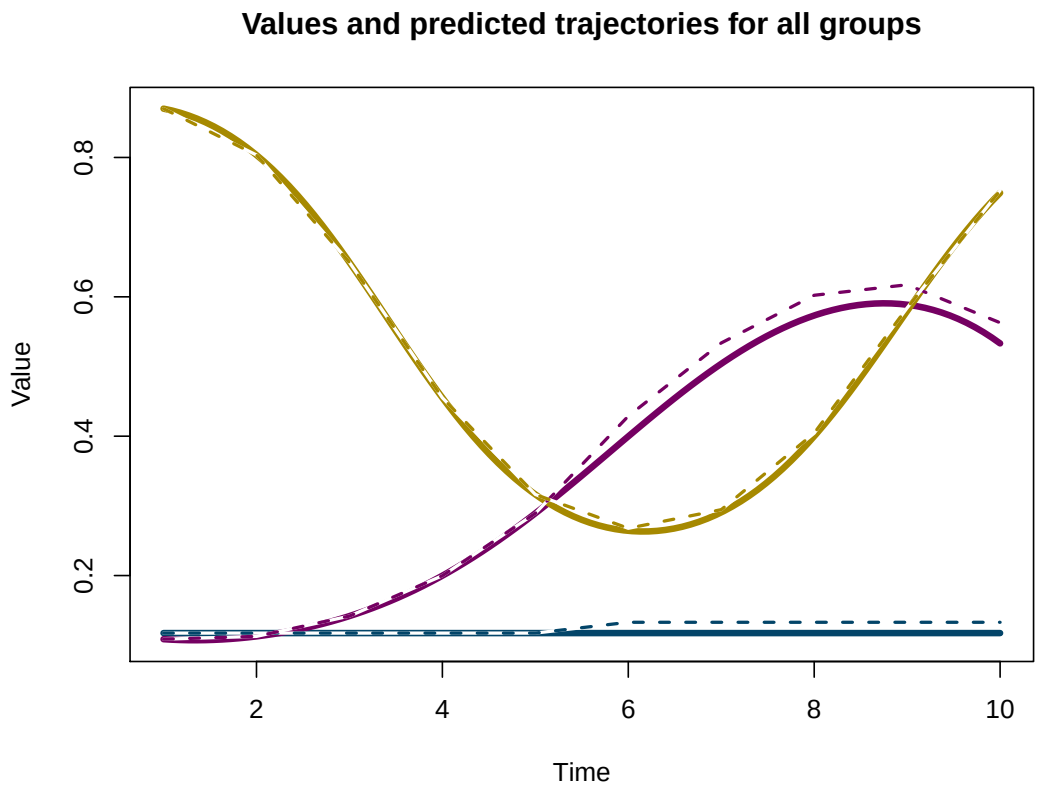
```
## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Logit
## Method : Expectation-maximization with IRLS
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept   -2.01725           NA          NA          NA
##               TCOV1    0.14188           NA          NA          NA
##
##       2   Intercept   -1.84956           NA          NA          NA
##              Linear   -0.42662           NA          NA          NA
##           Quadratic    0.18422           NA          NA          NA
##               Cubic   -0.01217           NA          NA          NA
##               TCOV1    0.11965           NA          NA          NA
##
##       3   Intercept    1.72138           NA          NA          NA
##              Linear    0.66141           NA          NA          NA
##           Quadratic    -0.5608           NA          NA          NA
##               Cubic     0.0827           NA          NA          NA
##           Quadratic   -0.00339           NA          NA          NA
##               TCOV1    0.01868           NA          NA          NA
## ---------------------------------------------------------------------
##       1         pi1     0.4187           NA          NA          NA
##       2         pi2    0.21781           NA          NA          NA
##       3         pi3     0.3635           NA          NA          NA
## ---------------------------------------------------------------------
## Likelihood : -2856.402
```

```
plot(solLTCOV, col = vcol)
```



If we want show the impact of a particular value of the time covariate in the trajectory, we can add this to the plot by `plotcov` option.
The fill line is the trajectory with the time covariate matrix and th dashed one show the impact on this trajectory of a particular value.

```
plot(solLTCOV, col = vcol, plotcov = c(0,0,0,0,0,1,1,1,1,1))
```



4 Censored Normal distribution

To illustrate this party we use artificial data contained in this package `CNORM_data01`.

5 Censored Normal model

We suppose that the variable Y_{it} is a censored variable, i.e. its values are between two numbers y_{min} and y_{max}.
We consider a variable Y^*_{it} that is normally distributed and such

$$y^*_{it} = f(a_{it}; \beta_k, \delta_k) + \epsilon_{it} = \beta_k A_{it} + \delta_k W_t + \epsilon_{it} \tag{3}$$

where $\epsilon_{it} \sim \mathcal{N}(0;\ \sigma)$, $A_{it} = (1, a_{it}, a^2_{it}, \cdots, a^{n_\beta - 1}_{it})^t$, $W_t = (w_{i1}, \cdots, w_{in_\delta})^t$, $\beta_k = (\beta_{k1}, \cdots, \beta_{kn_\beta})$ and $\delta_k = (\delta_{k1}, \cdots, \delta_{kn_\delta})$
Furthermore, we can link y^*_{it} to the observed and censored data y_{it}.

$$y_{it} = y_{min} \text{ if } y^*_{it} < y_{min} \tag{4}$$
$$y_{it} = y^*_{it} \text{ if } y_{min} \leq y^*_{it} \leq y_{max} \tag{5}$$
$$y_{it} = y_{max} \text{ if } y^*_{it} > y_{max} \tag{6}$$
$$\tag{7}$$

By asking $\mu_{ikt} = \beta_k A_{it} + \delta_k W_t$, we can write

$$P(Y_{it} = y_{it}|W_i = w_i, C_i = k) = \begin{cases} \Phi\left(\dfrac{y_{min} - \mu_{ikt}}{\sigma_k}\right) & \text{if } y^*_{it} < y_{min} \\ \dfrac{1}{\sigma_k}\phi\left(\dfrac{y_{it} - \mu_{ikt}}{\sigma_k}\right) & \text{if } y_{min} \leq y^*_{it} \leq y_{max} \\ 1 - \Phi\left(\dfrac{y_{max} - \mu_{ikt}}{\sigma_k}\right) & \text{if } y^*_{it} > y_{max} \end{cases} \tag{8}$$

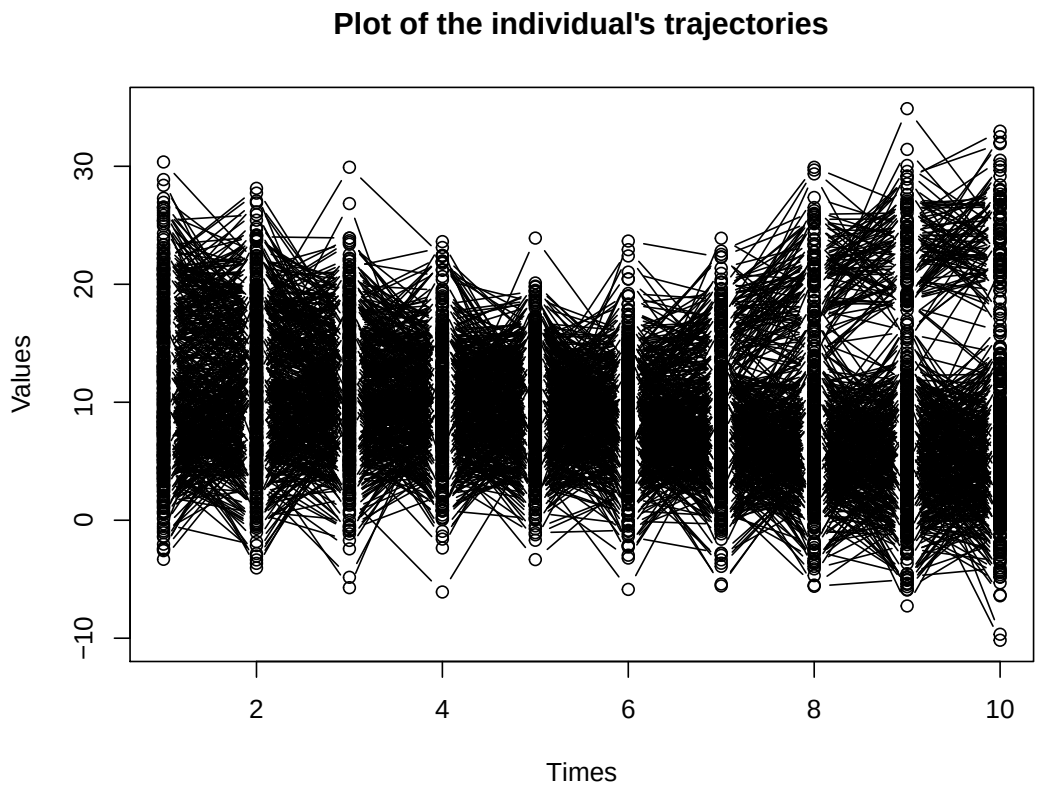
5.1 Parameters

Example
We use artificial data contained in the library. There are 500 longitudinal data generated by the parameters above :

- 3 groups ;
- probability is $\pi_1 = 0.32$, $\pi_2 = 0.54$ and $\pi_3 = 0.14$;
- period is 10 ;
- we use 3 polynomial shape
  1. degre 3 and $\beta_1 = (2.797, 8.809, -3.201, 0.463, -0.021)$;
  2. degre 4 and $\beta_2 = 7$;
  3. degre 0 and $\beta_3 = (19.545, -0.297, -0.407, 0.026)$.
- σ is the same for the 3 groups, $\sigma = 4$.

The data contain the time variable dependent Y_i in `data[,2:11]`, the time variable in `data[,12:21]`, the time dependent covariate in `data[,22:41]` and a covariate that influence the belonging probability in `data[,42:43]`.

- `data[,2:11]` is a matrix with real.
- `data[,12:21]` is a matrix with time 1 to 10.
- `data[,22:31]` is a time covariate that influence the shape fo trajectory. It is a matrix with 0 and 1 value, like the presence or not of a characteristic on the individual.
- `data[,32:41]` is a time covariate that influence the shape fo trajectory. It is a matrix with real.
- `data[,42:43]` is a matrix with real.



We use each method to fit the model. For all methods, we specify the number of group of our model, ng=3, the degree of the polynomial shape of the trajectory. Here we choice a line parallel to abscissa axis, a cubic polynomial and a quadric polynomial. So degree is a vector $(0, 3, 4)$.
We specify `hessian=TRUE` to ask the calculus of the hessian matrix.

For the Likelihood method we call `trajeR` with option `Method ="L"`. We specify if we want the same sigma in each group with the parameters `ssigma`. If it is `TRUE` then the algorithm search the same sigma in all group.

```
# Likelihood different sigma
solL = trajeR(Y = data[,2:11], A = data[,12:21],
              ng = 3, degre = c(0,3,4),
              Model = "CNORM", Method = "EM",
              hessian = TRUE, ssigma = FALSE)
```

```
solL

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Censored Normal
## Method : Expectation-maximization
```

```
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept     7.0494      0.08356    84.36189          0
##
##       2   Intercept   19.30454      0.60682    31.81271          0
##              Linear   -0.09315      0.45447    -0.20496    0.83761
##           Quadratic   -0.45614      0.09383     -4.8614          0
##               Cubic    0.02919      0.00563     5.18733          0
##
##       3   Intercept     1.6695      1.34917     1.23743    0.21599
##              Linear   10.11827      1.52068     6.65379          0
##           Quadratic   -3.70726      0.52667    -7.03905          0
##               Cubic    0.53764      0.07038      7.6394          0
##             Quartic   -0.02459      0.00318    -7.72826          0
## ---------------------------------------------------------------------
##       1      sigma1    3.95795      0.05919    66.86856          0
##       2      sigma2    4.11085      0.07048    58.32633          0
##       3      sigma3    4.00173      0.09153    43.72174          0
## ---------------------------------------------------------------------
##       1         pi1    0.45891      0.02247    20.42129          0
##       2         pi2    0.34901      0.02151    16.22397          0
##       3         pi3    0.19208      0.03111     6.17446          0
## ---------------------------------------------------------------------
## Likelihood : -14564.35
```

If we want force the use unique sigma for each groups we write `ssigma = TRUE`.

```
# Likelihood same sigma
solLs = trajeR(Y = data[,2:11], A = data[,12:21],
               ng = 3, degre = c(0,3,4),
               Model = "CNORM", Method = "L",
               ssigma = TRUE, hessian = TRUE)
```

```
solLs

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Censored Normal
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept    7.05315       0.0851    82.88488          0
##
##       2   Intercept   19.31443      0.07947   243.05329          0
##              Linear   -0.08916      0.10115    -0.88145    0.37812
```

```
##           Quadratic   -0.45758      0.03641   -12.56655          0
##               Cubic    0.02928      0.00277     10.5622          0
##
##       3   Intercept    1.67003      0.01209   138.08106          0
##              Linear   10.11777      0.02697   375.19145          0
##           Quadratic   -3.70707      0.04374   -84.75415          0
##               Cubic    0.53762      0.01283    41.89705          0
##             Quartic   -0.02459      0.00085    -28.7909          0
## ---------------------------------------------------------------------
##       1      sigma1    4.02025          NaN         NaN        NaN
##       2      sigma2    4.02025      0.01864   215.70733          0
##       3      sigma3    4.02025          NaN         NaN        NaN
## ---------------------------------------------------------------------
##       1         pi1    0.45979      0.06143    22.52374          0
##       2         pi2    0.34813      0.06501    17.00387          0
##       3         pi3    0.19208        0.076     6.72199          0
## ---------------------------------------------------------------------
## Likelihood : -14565.75
```

For use EM method we write the same syntax but change the variable `Method` to `EM`.

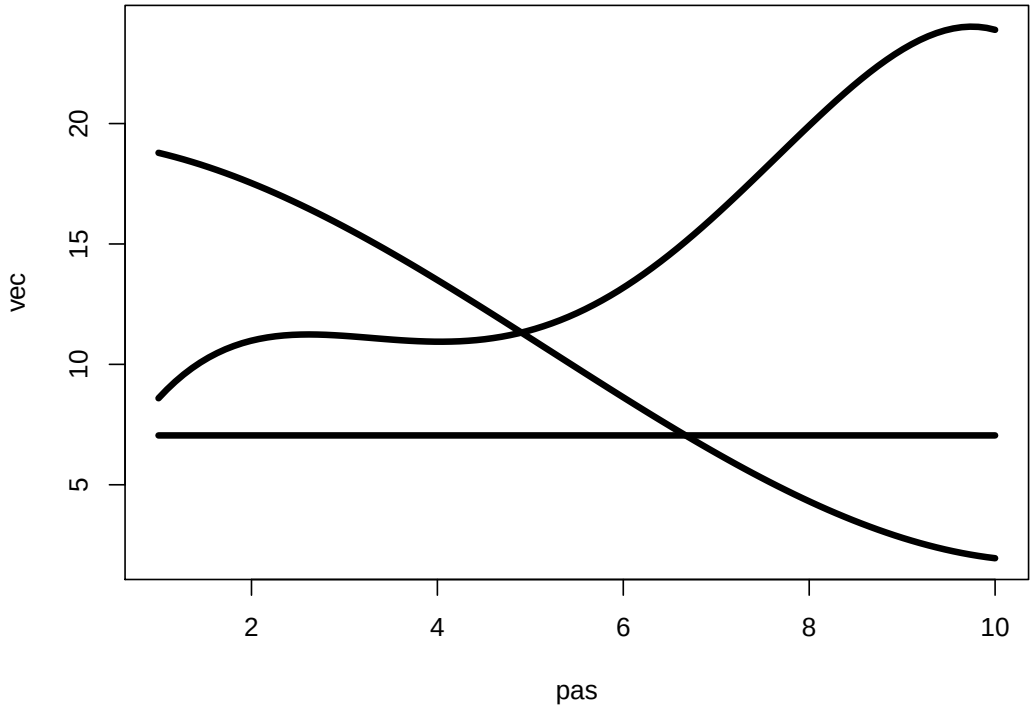
```r
# EM
solEM = trajeR(Y = data[,2:11], A = data[,12:21],
               ng = 3, degre = c(0,3,4),
               Model = "CNORM", Method = "EM",
               ssigma = FALSE, hessian = TRUE)
solEMs = trajeR(Y = data[,2:11], A = data[,12:21],
                ng = 3, degre = c(0,3,4),
                Model = "CNORM", Method = "EM",
                ssigma = TRUE, hessian = TRUE)
```

Different sigma		Same sigma	
SolL	SolEM	SolLs	SolEMs
Beta 1			
7.04940	7.04940	7.05315	7.05316
Beta 2			
19.30454	19.30454	19.31443	19.31467
-0.09315	-0.09315	-0.08916	-0.08935
-0.45614	-0.45614	-0.45758	-0.45753
0.02919	0.02919	0.02928	0.02927
Beta 3			
1.66950	1.66950	1.67003	1.66950
10.11827	10.11827	10.11777	10.11834
-3.70726	-3.70726	-3.70707	-3.70726
0.53764	0.53764	0.53762	0.53764
-0.02459	-0.02459	-0.02459	-0.02459
Sigma			
3.95795	3.95795	4.02025	4.02028
4.11085	4.11085	4.02025	4.02028
4.00173	4.00173	4.02025	4.02028
Pi			
0.37570	0.45891	0.45979	0.45978
0.33660	0.34901	0.34813	0.34814
0.28771	0.19208	0.19208	0.19207

5.2 Plot

We plot only the trajectories on a graph. We have just to use the native plot function. It use the class of the object to draw the polynomial shape. The values in abscissa are the first row of the time variable.

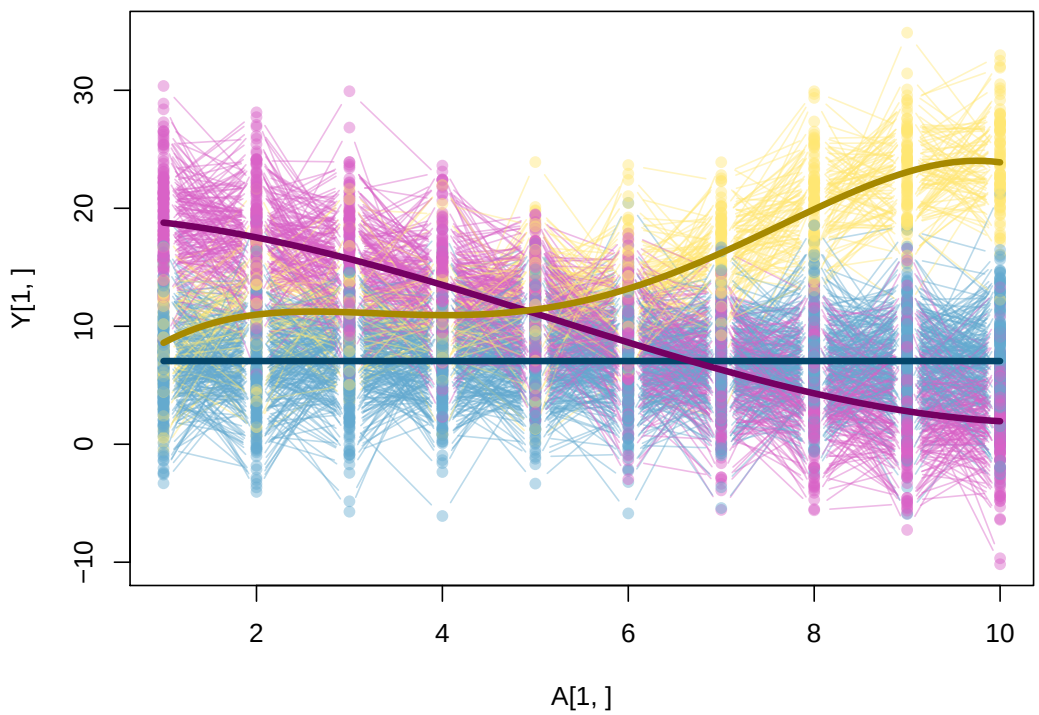
```
plot(solL)
```



We can add longitudinal data to this graph. If we want this on the plot we have to specify `Y` and `A` in the function `plot()`.
By default colors are gray scale, but we can specify colors we want.

```
# colour's defintion
trans = "70"
col1 = "#034569"
col1.1 = paste0("#64AAD0", trans)
col2 = "#750062"
col2.1 = paste0("#D962C7", trans)
col3 = "#A68900"
col3.1 = paste0("#FFE773", trans)
cols1 = c(col1.1, col2.1, col3.1)
cols2 = c(col1, col2, col3)
vcol = c(cols1, cols2)

plot(solEM, Y = data[,2:11], A = data[,12:21], col = vcol)
```



5.3 Adding covariate

In the same way that in the section below we can add covariate in the calculus of the parameters with or without option same sigma.

If we add risk variable we use option `Risk =`.

```
solLRisks = trajeR(Y = data[,2:11], A = data[,12:21], Risk = data[,42:43],
                   ng = 3, degre = c(0,3,4),
                   Model = "CNORM", Method = "L",
                   ssigma = TRUE, hessian = TRUE)
```

```
solLRisks

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Censored Normal
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                param.=0
```

```
## ---------------------------------------------------------------------
##       1   Intercept    7.05305      0.04176   168.90808          0
##
##       2   Intercept   19.31485      0.11614   166.30125          0
##              Linear   -0.09043      0.11968    -0.75564     0.4499
##           Quadratic   -0.45729      0.04884    -9.36403          0
##               Cubic    0.02926      0.03016      0.9701    0.33204
##
##       3   Intercept    1.66973      0.03157     52.8925          0
##              Linear   10.11813      0.08516   118.81332          0
##           Quadratic   -3.70714      0.07949   -46.63367          0
##               Cubic    0.53762      0.10117     5.31376          0
##             Quartic   -0.02459      0.03642    -0.67503    0.49969
## ---------------------------------------------------------------------
##       1      sigma1    4.02023      0.00277   1450.09467          0
##       2      sigma2    4.02023      0.01226    327.8234          0
##       3      sigma3    4.02023      0.02714    148.1213          0
## ---------------------------------------------------------------------
##       1    Baseline          0      0.04392    36.76149          0
##
##       2   Intercept   -0.65926          NaN         NaN        NaN
##                 V42    0.19556      0.01854    59.05037          0
##                 V43    0.57349          NaN         NaN        NaN
##
##       3   Intercept   -1.18455       0.0403     10.6709          0
##                 V42    0.10637      0.13123     7.66374          0
##                 V43    0.52564      0.13637      8.5007          0
## ---------------------------------------------------------------------
## Likelihood : -14563.99
```

Different sigma		Same sigma	
SolLRisk	SolEMRisk	SolLRisks	SolEMRisks
Beta 1			
7.04921	7.04923	7.05305	7.05306
Beta 2			
19.30527	19.30497	19.31485	19.31517
-0.09488	-0.09451	-0.09043	-0.09075
-0.45575	-0.45584	-0.45729	-0.45723
0.02917	0.02917	0.02926	0.02926
Beta 3			
1.68068	1.66955	1.66973	1.66955
10.10606	10.11828	10.11813	10.11834
-3.70319	-3.70722	-3.70714	-3.70723
0.53712	0.53763	0.53762	0.53763
-0.02456	-0.02459	-0.02459	-0.02459
Sigma			
3.95758	3.95767	4.02023	4.02025
4.11102	4.11119	4.02023	4.02025
4.00154	4.00162	4.02023	4.02025
Theta - First group 0			
-0.65610	-0.65570	-0.65926	-0.65904
0.19568	0.19567	0.19556	0.19531
0.57614	0.57547	0.57349	0.57361
-1.18333	-1.18319	-1.18455	-1.18428
0.10695	0.10679	0.10637	0.10630
0.52682	0.52683	0.52564	0.52553

If we add time dependent covariate we use option `TCOV`.

```
solLTCOV2 = trajeR(Y = data[,2:11], A = data[,12:21], TCOV = data[,22:41],
                   ng = 3, degre = c(0,3,4),
                   Model = "CNORM", Method = "L",
                   ssigma = FALSE, hessian = TRUE)
```

```
solLTCOV2

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Censored Normal
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ------------------------------------------------------------------
##       1   Intercept    6.75745      0.08309    81.32696          0
##               TCOV1   -0.00358      0.16289    -0.02195    0.98249
```

```
##                TCOV2    0.58151      0.06557     8.86839          0
##
##       2   Intercept   19.45552      0.07795   249.59559          0
##               Linear   -0.09592      0.10136    -0.94628    0.34405
##            Quadratic   -0.45507      0.03622   -12.56502          0
##                Cubic     0.0291      0.00277    10.51302          0
##                TCOV1   -0.17051      0.19635     -0.8684    0.38522
##                TCOV2   -0.13303      0.05521    -2.40954    0.01601
##
##       3   Intercept    1.79828      0.06183    29.08577          0
##               Linear   10.13362      0.03745   270.57751          0
##            Quadratic   -3.71451      0.07286   -50.98126          0
##                Cubic    0.53887      0.01732    31.10904          0
##              Quartic   -0.02465      0.00106   -23.18069          0
##                TCOV1    0.08064      0.21207     0.38024    0.70378
##                TCOV2    -0.3598        0.038    -9.46729          0
## ---------------------------------------------------------------------
##       1      sigma1    3.95415      0.01531   258.26618          0
##       2      sigma2    4.10997      0.01714   239.80629          0
##       3      sigma3    4.00024      0.02353   169.98647          0
## ---------------------------------------------------------------------
##       1         pi1    0.45881       0.0617    22.38634          0
##       2         pi2    0.34911      0.06516    17.00593          0
##       3         pi3    0.19207       0.0761     6.70953          0
## ---------------------------------------------------------------------
## Likelihood : -14561.48
```

Here we can use all method and with or without same sigma.

```
solLTCOV2s = trajeR(data[,2:11], data[,12:21], TCOV = data[,22:41], ng = 3, degre=c(0,3,4),
                    Model="CNORM", Method = "L",
                    ssigma = TRUE, hessian = TRUE)
solEMTCOV2 = trajeR(data[,2:11], data[,12:21], TCOV = data[,22:41], ng = 3, degre=c(0,3,4),
                   Model="CNORM", Method = "EM",
                   ssigma = FALSE, hessian = TRUE)
solEMTCOV2s = trajeR(data[,2:11], data[,12:21], TCOV = data[,22:41], ng = 3, degre=c(0,3,4),
                   Model="CNORM", Method = "EM",
                   ssigma = TRUE, hessian = TRUE)
```

We have

Different sigma		Same sigma	
SolLTCOV2	SolEMCOV2	SolLTCOV2s	SolEMCOV2s
Beta 1			
6.75745	6.75763	6.76374	6.76375
Beta 2			
19.45552	19.45636	19.46899	19.46852
-0.09592	-0.09667	-0.09357	-0.09328
-0.45507	-0.45490	-0.45619	-0.45624
0.02910	0.02909	0.02917	0.02917
Beta 3			
1.79828	1.81447	1.81399	1.81446
10.13362	10.11476	10.11540	10.11485
-3.71451	-3.70808	-3.70827	-3.70809
0.53887	0.53803	0.53806	0.53803
-0.02465	-0.02462	-0.02462	-0.02462
Sigma			
3.95415	3.95414	4.01798	4.01801
4.10997	4.11016	4.01798	4.01801
4.00024	4.00019	4.01798	4.01801
Delta 1			
-0.00358	-0.00354	-0.00768	-0.00762
0.58151	0.58136	0.58100	0.58095
Delta 2			
-0.17051	-0.17068	-0.16727	-0.16724
-0.13303	-0.13289	-0.13786	-0.13777
Delta 3			
0.08064	0.08049	0.08040	0.08044
-0.35980	-0.35939	-0.35958	-0.35934
Pi			
0.45881	0.45879	0.45968	0.45970
0.34911	0.34913	0.34824	0.34823
0.19207	0.19208	0.19208	0.19207

6 Censored Data

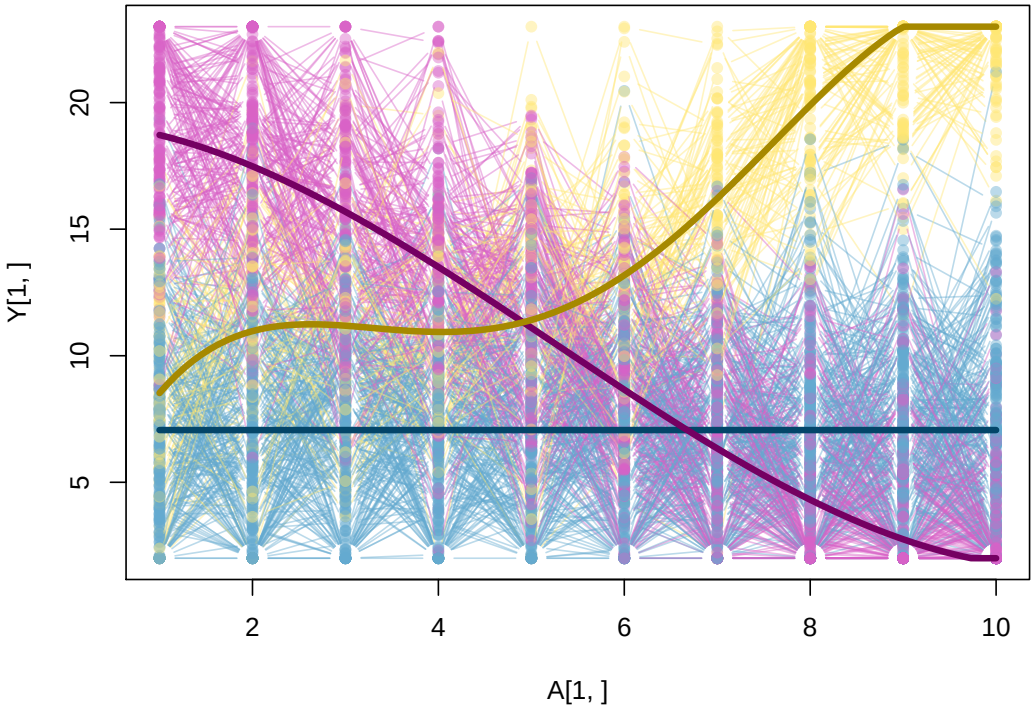
We have modified the previous data in such way that they become censored data. All values upper to 23 become 23 and those smaller to 2 become 2.
So we obtained data that follow a censored normal distribution. If we want take into consideration this fact,

```
solLC = trajeR(Y = dataC[,2:11], A = dataC[,12:21],
               ng = 3, degre = c(0,3,4),
               Model="CNORM", Method = "L",
               ssigma = FALSE, hessian = TRUE,
               ymin=2, ymax=23)
```

```
solLC

## Call TrajeR with 3 groups and a 0,3,4 degrees of polynomial shape of trajectory.
## Model : Censored Normal
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept    7.06349      0.08309    85.01078          0
##
##       2   Intercept   19.22938      0.08478   226.81088          0
##              Linear   -0.08816      0.10743    -0.82057    0.41193
##           Quadratic   -0.44944      0.03883   -11.57498          0
##               Cubic    0.02842      0.00297     9.58168          0
##
##       3   Intercept    1.46919       0.0892    16.46982          0
##              Linear   10.30385      0.12286    83.86753          0
##           Quadratic   -3.76773          NaN         NaN        NaN
##               Cubic    0.54588          NaN         NaN        NaN
##             Quartic   -0.02499          NaN         NaN        NaN
## ---------------------------------------------------------------------
##       1      sigma1     3.9223      0.01609   243.72812          0
##       2      sigma2    4.10489      0.01955    209.9628          0
##       3      sigma3    4.00575      0.00694   576.81651          0
## ---------------------------------------------------------------------
##       1         pi1    0.45833       0.0614    22.47794          0
##       2         pi2    0.34959      0.06512    17.03599          0
##       3         pi3    0.19209      0.07573     6.74124          0
## ---------------------------------------------------------------------
## Likelihood : -13317.83

plot(solLC, Y = dataC[,2:11], A = dataC[,12:21], col = vcol)
```

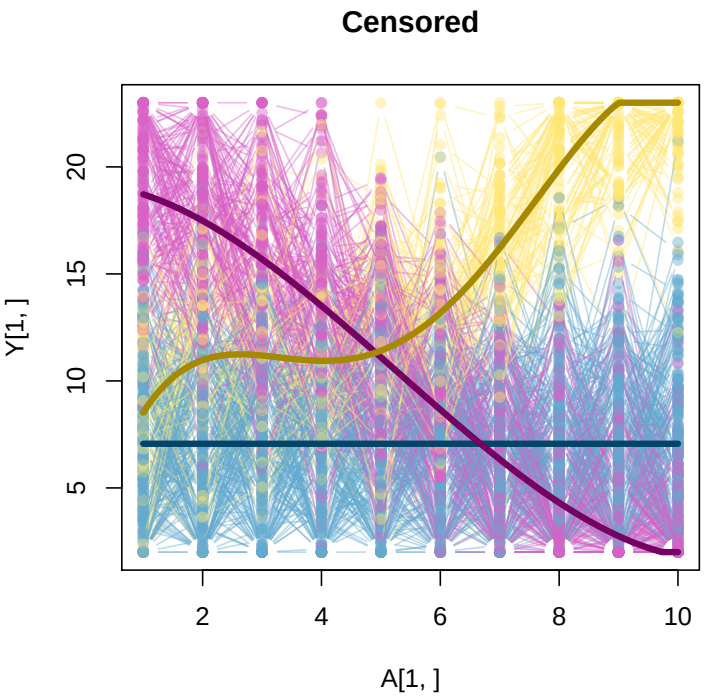


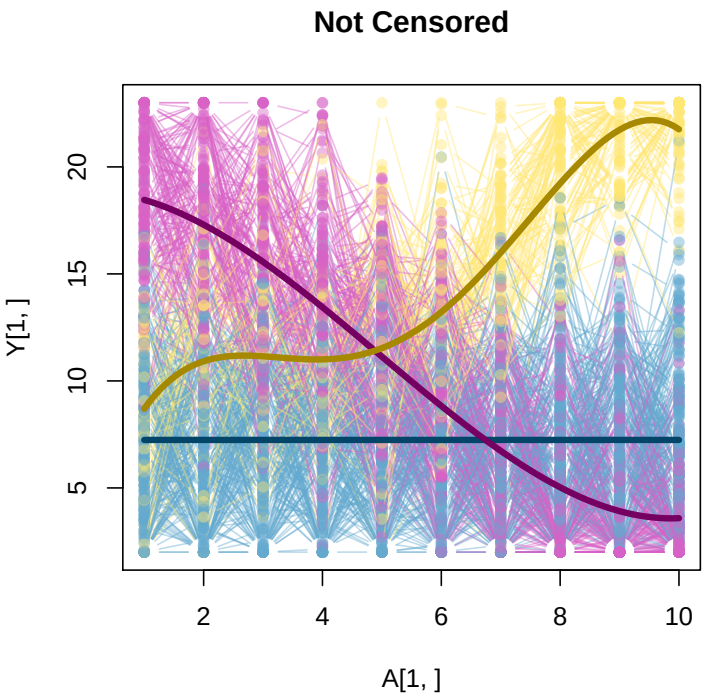
If we search the solution without considering the fact that the data are not censored, we find parameters that are wrong. This amounts to considering the maximum and minimum are really maximum and minimum while the real values are omitted. To illustrate this fact we compute the parameters without censored.

Censored	Not Censored
Beta 1	
7.0634900	7.2424800
Beta 2	
19.2293800	18.8466500
-0.0881600	0.0639900
-0.4494400	-0.4835500
0.0284200	0.0324500
Beta 3	
1.4691900	2.3253200
10.3038500	9.3139500
-3.7677300	-3.4181600
0.5458800	0.5013300
-0.0249900	-0.0233200
Sigma	
3.9223000	3.5980600
4.1048900	3.5482400
4.0057500	3.6012300
Pi	
0.4583245	0.4588787
0.3495887	0.3489176
0.1920868	0.1922037

We can see the difference between the two situations in the graphic above. The right one is calculated by considering censored data and it is close to the right solution while the left one consider not censored data. The right trajectories consider the missing values by continuing up while the left one consider the max and min like real values and the trajectories is smaller.

```
par(mfrow=c(1,2))
plot(solLC, Y = dataC[,2:11], A = dataC[,12:21], col = vcol, main="Censored")
plot(solLnC, Y = dataC[,2:11], A = dataC[,12:21], col = vcol, main="Not Censored" )
```





For all the method we have

```
solLCs = trajeR(Y = dataC[,2:11], A = dataC[,12:21],
                ng = 3, degre = c(0,3,4),
                Model="CNORM", Method = "L",
                ssigma = TRUE, hessian = FALSE,
                ymin = 2,  ymax = 23)
solEMC = trajeR(Y = dataC[,2:11], A = dataC[,12:21],
                ng = 3, degre = c(0,3,4),
                Model="CNORM", Method = "EM",
                ssigma = FALSE, hessian = FALSE,
                ymin = 2,  ymax = 23)
solEMCs = trajeR(Y = dataC[,2:11], A = dataC[,12:21],
                 ng = 3, degre = c(0,3,4),
                 Model="CNORM", Method = "EM",
                 ssigma = TRUE, hessian = FALSE,
                 ymin = 2,  ymax = 23)
```

Different sigma		Same sigma	
SolLC	SolEMC	SolLCs	SolEMCs
Beta 1			
7.06349	7.06352	7.06263	7.06261
Beta 2			
19.22938	19.23116	19.22999	19.23109
-0.08816	-0.08916	-0.08092	-0.08164
-0.44944	-0.44929	-0.45150	-0.45135
0.02842	0.02841	0.02860	0.02859
Beta 3			
1.46919	1.47075	1.47182	1.47257
10.30385	10.30213	10.30113	10.30028
-3.76773	-3.76720	-3.76673	-3.76648
0.54588	0.54581	0.54574	0.54571
-0.02499	-0.02499	-0.02498	-0.02498
Sigma			
3.92230	3.92236	3.99933	3.99933
4.10489	4.10491	3.99933	3.99933
4.00575	4.00573	3.99933	3.99933
Pi			
0.45833	0.45829	0.45939	0.45937
0.34959	0.34961	0.34855	0.34855
0.19209	0.19210	0.19207	0.19208

We can add time covariate to the censored data.

```
solLCTCOV = trajeR(Y = dataC[,2:11], A = dataC[,12:21], TCOV = dataC[,22:31],
                   ng = 3, degre = c(0,3,4),
                   Model = "CNORM", Method = "L",
                   ssigma = FALSE, hessian = FALSE,
                   ymin = 2,  ymax = 23)
solLCTCOVs = trajeR(Y = dataC[,2:11], A = dataC[,12:21], TCOV = dataC[,22:31],
                    ng = 3, degre = c(0,3,4),
                    Model = "CNORM", Method = "L",
                    ssigma = TRUE, hessian = FALSE,
                    ymin = 2,  ymax = 23)
solEMCTCOV = trajeR(Y = dataC[,2:11], A = dataC[,12:21], TCOV = dataC[,22:31],
                    ng = 3, degre = c(0,3,4),
                    Model = "CNORM", Method = "EM",
                    ssigma = TRUE, hessian = FALSE,
                    ymin = 2,  ymax = 23)
solEMCTCOVs = trajeR(Y = dataC[,2:11], A = dataC[,12:21], TCOV = dataC[,22:31],
                     ng = 3, degre = c(0,3,4),
                     Model = "CNORM", Method = "EM",
                     ssigma = FALSE, hessian = FALSE,
                     ymin = 2,  ymax = 23)
```

Different sigma		Same sigma	
SolLCTCOV	SolEMCTCOV	SolLCTCOVs	SolEMCTCOVs
Beta 1			
7.05896	7.06197	7.06096	7.06301
Beta 2			
19.32135	19.22620	19.31753	19.22625
-0.08348	-0.08183	-0.07488	-0.08963
-0.44995	-0.45094	-0.45224	-0.44880
0.02843	0.02854	0.02862	0.02836
Beta 3			
1.41791	1.47169	1.41970	1.46970
10.27967	10.30199	10.27817	10.30404
-3.75946	-3.76710	-3.75892	-3.76789
0.54479	0.54578	0.54472	0.54589
-0.02494	-0.02498	-0.02494	-0.02499
Sigma			
3.92169	4.00062	3.99867	3.92158
4.10424	4.00062	3.99867	4.10881
4.00525	4.00062	3.99867	4.00763
Delta 1			
0.00795	0.00089	0.00218	0.00389
Delta 2			
-0.20531	-0.09786	-0.20025	-0.10035
Delta 3			
0.13639	0.06155	0.13628	0.06160
Pi			
0.45819	0.45918	0.45927	0.45808
0.34971	0.34874	0.34864	0.34983
0.19210	0.19208	0.19209	0.19209

7 Zero Inflated Poisson

The ZIP (Zero Inflated Poisson) employ two different process : a binary distribution that generate structural zero, i.e. the excess zeros, and and Poisson distribution that generate the counts.

Under this model, zero can occur for two reasons. The first is the counts is equal to zero with a probability $P(Y_{it} = 0)$ for $Y_{it} \sim \mathcal{P}(\lambda_{ikt})$ and the second become of the fact the binary produce zero values with a probability ρ_{ikt}.

We have

$$P\left(Y_{it} = y_{it} | W_i = wi, C_i = ci\right) = \begin{cases} \rho_{ikt} + (1-\rho_{ikt})e^{-\lambda_{ikt}}, \ y_{it} = 0 \\ (1-\rho_{ikt})\frac{\lambda_{ikt}^{y_{it}} e^{-\lambda_{ikt}}}{y_{it}!}, \ y_{it} > 0 \end{cases} \tag{9}$$

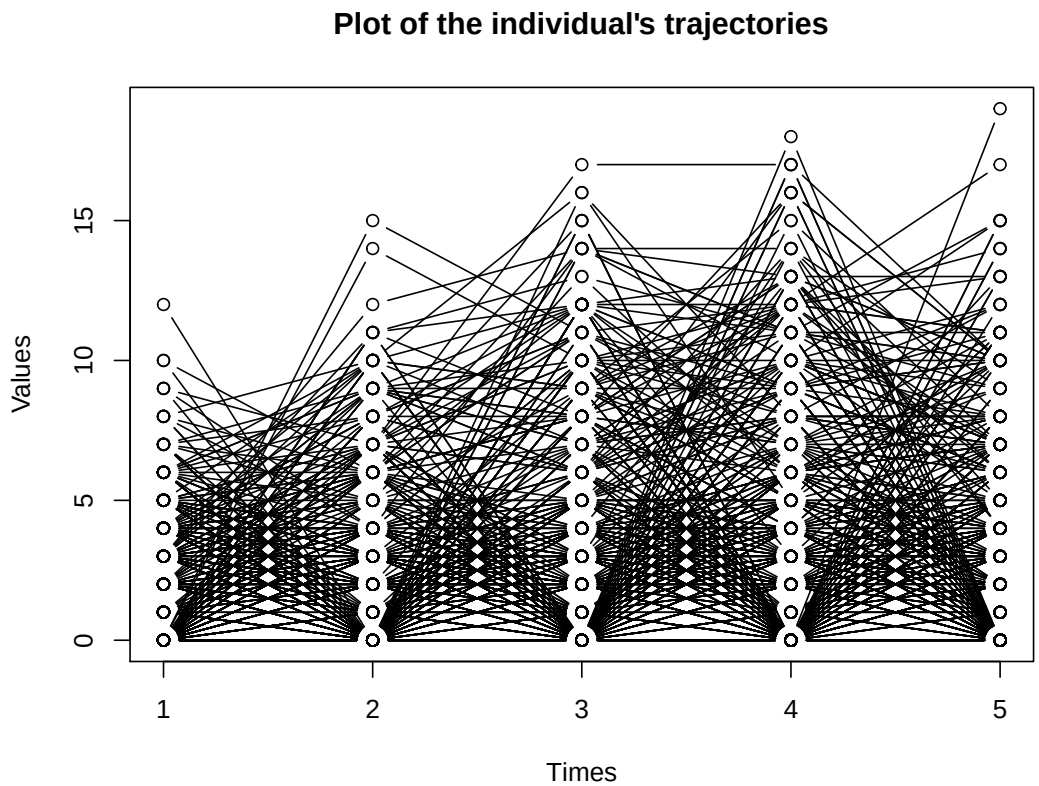
7.1 Parameters

Example
We use artificial data contained in the library. There are 500 longitudinal data generated by the parameters above :

- 2 groups ;
- probability is $\pi_1 = 0.3$ and $\pi_2 = 0.7$;
- period is 5 ;
- we use 2 polynomial shape to calculate the parameters λ of the Poisson state
  1. degree 2 and $\beta_1 = (1.2, 0.5, -0.06)$;
  2. degree 2 and $\beta_2 = (0.89, 0.01, 0.01)$;
- we use 2 polynomial shape to calculate the parameters ν of the zero state
  1. degree 1 and $\nu_1 = (-0.2, -0.1)$;
  2. degree 1 and $\nu_2 = (-1, 0)$;

The data contained the time variable dependent Y_i in `dataZIP[,2:6]`, the time variable in `dataZIP[,7:11]`, the time covariate in `dataZIP[,13:17]` and a covariate that influence the belonging probability in `dataZIP[,12]`.

- `dataZIP[,2:6]` is a matrix with real.
- `dataZIP[,7:11]` is a matrix with time 1 to 10.
- `dataZIP[,13:17]` is a time covariate that influence the shape of the trajectories. It is a matrix with 0 and 1 value, like the presence or not of a characteristic on the individual.
- `dataZIP[,12]` is a matrix with real.



We use each method to fit the model. For all methods, we specify the number of group of our model, ng=2 and the degree of the polynomial shape of the trajectories for the two state. For the Poisson state we choice a 3 degree polynomial and for the zero state a 2 degree polynomial.
Thus for β degree is a vector $(3, 3)$ and for ν degree is a vector $(2, 2)$.
We specify `hessian=TRUE` to ask the calculus of the hessian matrix.

For the Likelihood method we call `trajeR` with option `Method ="L"`.

```
# Likelihood
solL = trajeR(Y = data[,2:6], A = data[,7:11],
              ng = 2, degre=c(2,2), degre.nu = c(1,1),
              Model="ZIP", Method = "L", hessian = TRUE)
```

```
solL

## Call TrajeR with 2 groups and a 2,2 degrees of polynomial shape of trajectory.
## Model : Zero Inflated Poisson
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
```

```
##                                                  param.=0
## ---------------------------------------------------------------------
##       1   Intercept    0.90784      0.09589     9.46764          0
##              Linear    0.01872      0.07032     0.26615    0.79014
##           Quadratic    0.00618      0.01131     0.54684    0.58454
##                Nu11   -1.08146      0.16018    -6.75139          0
##                Nu12    0.00453      0.04689     0.09663    0.92303
##
##       2   Intercept    1.04864      0.09415    11.13833          0
##              Linear    0.61513      0.06423      9.5775          0
##           Quadratic   -0.07774      0.00998    -7.79273          0
##                Nu21    0.03158      0.16749     0.18852    0.85048
##                Nu22   -0.16774      0.05183     -3.2362    0.00123
## ---------------------------------------------------------------------
##       1         pi1    0.65134      0.04936    26.59111          0
##       2         pi2    0.34866      0.04936    13.92996          0
## ---------------------------------------------------------------------
## Likelihood : -5162.009
```

For use EM method we write the same syntax but change the variable `Method` to `EM` or `EMIRLS`. In the first case, we find the parameters with quasi newton method and in the second one with Iterative Reweighted Least Square.

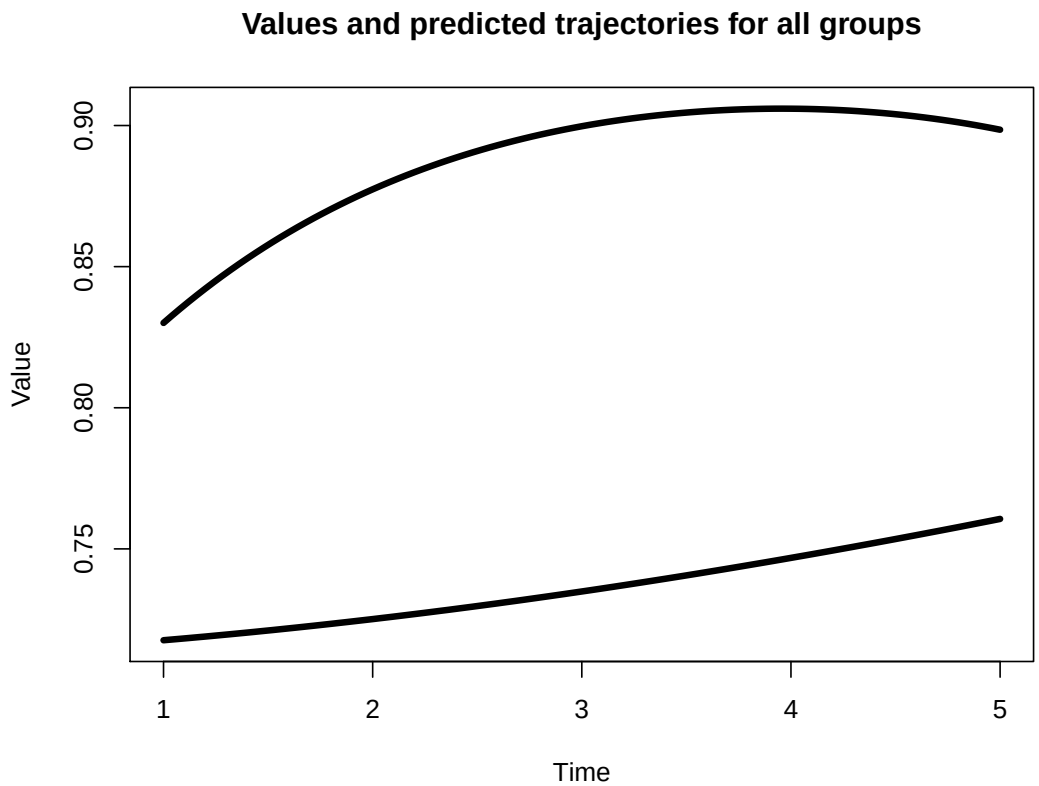
```
# EM
solEM = trajeR(Y = data[,2:6], A = data[,7:11],
               ng = 2, degre=c(2,2), degre.nu = c(1,1),
               Model="ZIP", Method = "EM", hessian = TRUE)
solEMIRLS = trajeR(Y = data[,2:6], A = data[,7:11],
                   ng = 2, degre=c(2,2), degre.nu = c(1,1),
                   Model="ZIP", Method = "EMIRLS", hessian = TRUE)
```

SolL		SolEM		SolEMIRLS	
parameters	sd	parameters	sd	parameters	sd
Beta 1					
0.90784	0.09589	0.90737	0.07865	0.90782	0.07864
0.01872	0.07032	0.01898	0.05853	0.01872	0.05853
0.00618	0.01131	0.00615	0.00947	0.00618	0.00947
Beta 2					
1.04864	0.09415	1.04879	0.07417	1.04873	0.07417
0.61513	0.06423	0.61503	0.04979	0.61507	0.04979
-0.07774	0.00998	-0.07773	0.00769	-0.07773	0.00769
Nu 1					
-1.08146	0.16018	-1.08176	0.16536	-1.08146	0.16534
0.00453	0.04689	0.00461	0.04874	0.00454	0.04874
Nu 2					
0.03158	0.16749	0.03159	0.18424	0.03162	0.18424
-0.16774	0.05183	-0.16775	0.05663	-0.16776	0.05663
Pi					
0.65134	0.04936	0.65132	0.02151	0.65132	0.02151
0.34866	0.04936	0.34868	0.02151	0.34868	0.02151

7.2 Plot

We plot only the trajectories on the graph. We have just to use the native plot function. It use the class of the object to draw the polynomial shape. The values in abscissa are the first row of the time variable.

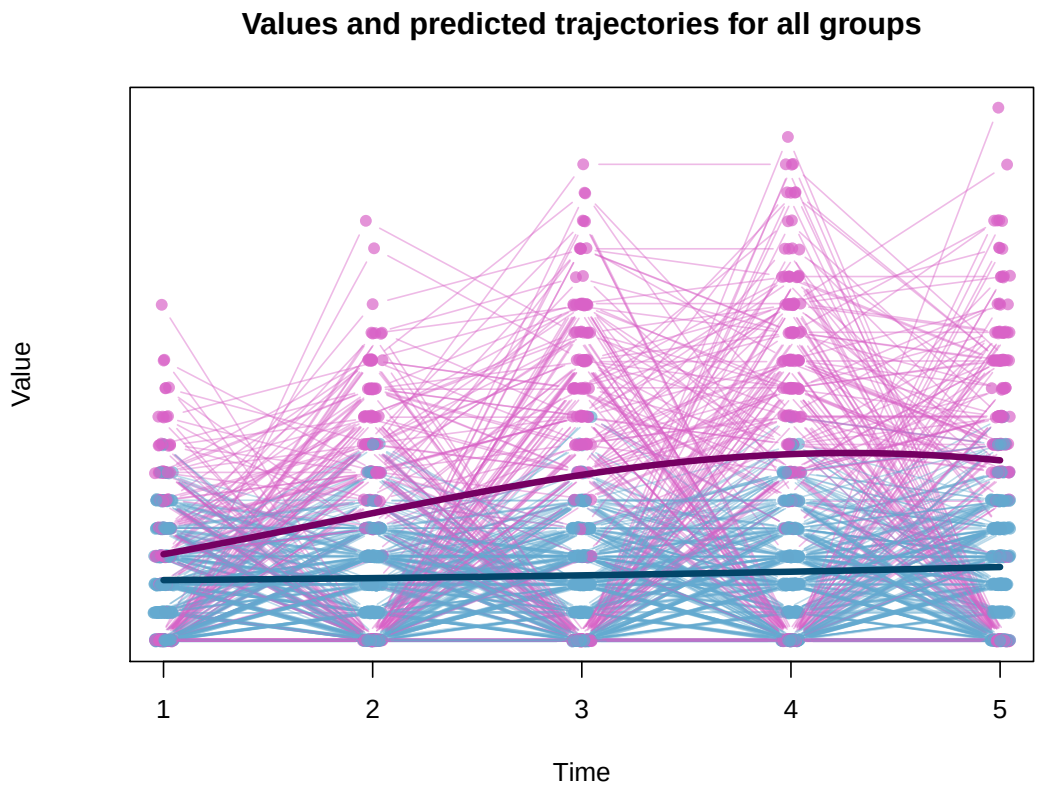
```
plot(solL)
```



We can add longitudinal data to this graph. If we want this on the plot we have to specify `Y` and `A` in the function `plot()`.
By default colors are gray scale, but we can specify colors we want.

```
# colour's defintion
trans = "70"
col1 = "#034569"
col1.1 = paste0("#64AAD0", trans)
col2 = "#750062"
col2.1 = paste0("#D962C7", trans)
cols1 = c(col1.1, col2.1)
cols2 = c(col1, col2)
vcol = c(cols1, cols2)

plot(solEM, Y = dataZIP[,2:6], A = dataZIP[,7:11], dec = 5, col = vcol)
```



7.3 Adding covariate

In the same way that in the section below we can add covariate in the calculus of the parameters.

If we add risk variable we use option `Risk =`.

```
solLRisk = trajeR(Y = data[,2:6], A = data[,7:11], Risk = data[,12],
                  ng = 2, degre=c(2,2), degre.nu = c(1,1),
                  Model="ZIP", Method = "L", hessian = TRUE)
```

```
solLRisk

## Call TrajeR with 2 groups and a 2,2 degrees of polynomial shape of trajectory.
## Model : Zero Inflated Poisson
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                param.=0
## ---------------------------------------------------------------------
##       1   Intercept     0.9073      0.09888      9.1762          0
```

```
##              Linear    0.01913      0.16707      0.1145    0.90885
##           Quadratic    0.00609      0.09582     0.06357    0.94932
##                Nu11   -1.08261       0.0642   -16.86232          0
##                Nu12    0.00465      0.00997     0.46656    0.64086
##
##       2   Intercept    1.04857       0.0703    14.91602          0
##              Linear     0.6149      0.01131    54.38549          0
##           Quadratic    -0.0777      0.09411    -0.82564    0.40909
##                Nu21    0.03215      0.16036     0.20047    0.84113
##                Nu22   -0.16788      0.04693    -3.57735    0.00035
## ---------------------------------------------------------------------
##       1    Baseline          0       0.1674     8.41028          0
##
##       2   Intercept   -0.81584      0.09888     5.98811          0
##                   X     0.3763      0.16707     7.11177          0
## ---------------------------------------------------------------------
## Likelihood : -5161.373
```

We can use all method too.

```
solEMRisk = trajeR(Y = data[,2:6], A = data[,7:11], Risk = data[,12],
                   ng = 2, degre=c(2,2), degre.nu = c(1,1),
                   Model="ZIP", Method = "EM", hessian = TRUE)
solEMIRLSRisk = trajeR(Y = data[,2:6], A = data[,7:11], Risk = data[,12],
                       ng = 2, degre=c(2,2), degre.nu = c(1,1),
                       Model="ZIP", Method = "EMIRLS", hessian = TRUE)
```

We have

SolLRisk		SolEMRisk		SolEMIRLSRisk	
parameters	sd	parameters	sd	parameters	sd
Beta 1					
0.90730	0.09888	0.90727	0.0786111	0.90742	0.07861
0.01913	0.16707	0.01917	0.0585169	0.01908	0.05852
0.00609	0.09582	0.00608	0.0094663	0.00610	0.00947
Beta 2					
1.04857	0.07030	1.04844	0.0746396	1.04842	0.07464
0.61490	0.01131	0.61493	0.0497852	0.61501	0.04978
-0.07770	0.09411	-0.07770	0.0076923	-0.07772	0.00769
Nu 1					
-1.08261	0.06420	-1.08244	0.1655128	-1.08237	0.16551
0.00465	0.00997	0.00464	0.0487706	0.00462	0.04877
Nu 2					
0.03215	0.16036	0.03213	0.1842317	0.03215	0.18423
-0.16788	0.04693	-0.16783	0.0566009	-0.16784	0.05660
Theta					
0.00000	0.16740	0.00000	0.0000000	0.00000	0.00000
0.00000	0.05179	0.00000	0.0000000	0.00000	0.00000
0.81584	0.09888	-0.81595	0.2025596	0.81597	0.20256
-0.37630	0.16707	0.37653	0.3347386	-0.37652	0.33474

7.4 Adding time depdendent covariate

If we add time dependent covariate we use option `TCOV`.

```
solLTCOV = trajeR(Y = data[,2:6], A = data[,7:11], TCOV = data[,13:17],
                  ng = 2, degre=c(2,2), degre.nu = c(1,1),
                  Model="ZIP", Method = "L", hessian = TRUE)
```

```
solLTCOV

## Call TrajeR with 2 groups and a 2,2 degrees of polynomial shape of trajectory.
## Model : Zero Inflated Poisson
## Method : Likelihood
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept     0.8808      0.09727     9.05511          0
##              Linear    0.01535      0.07035     0.21818    0.82731
##           Quadratic    0.00677      0.01131     0.59868    0.54944
##                Nu11   -1.07073      0.15947    -6.71429          0
##                Nu12    0.00232      0.04674     0.04971    0.96036
##               TCOV1    0.06373      0.03813     1.67142    0.09476
```

```
##
##       2   Intercept    1.04296      0.09512    10.96488          0
##              Linear    0.61468      0.06423      9.5696          0
##           Quadratic   -0.07766      0.00998    -7.78283          0
##                Nu21    0.02787      0.16794     0.16593    0.86822
##                Nu22   -0.16707      0.05197    -3.21474    0.00132
##               TCOV1    0.01298      0.03174     0.40904    0.68254
## ---------------------------------------------------------------------
##       1         pi1    0.65197      0.04946    26.56448          0
##       2         pi2    0.34803      0.04946    13.87274          0
## ---------------------------------------------------------------------
## Likelihood : -5160.531
```

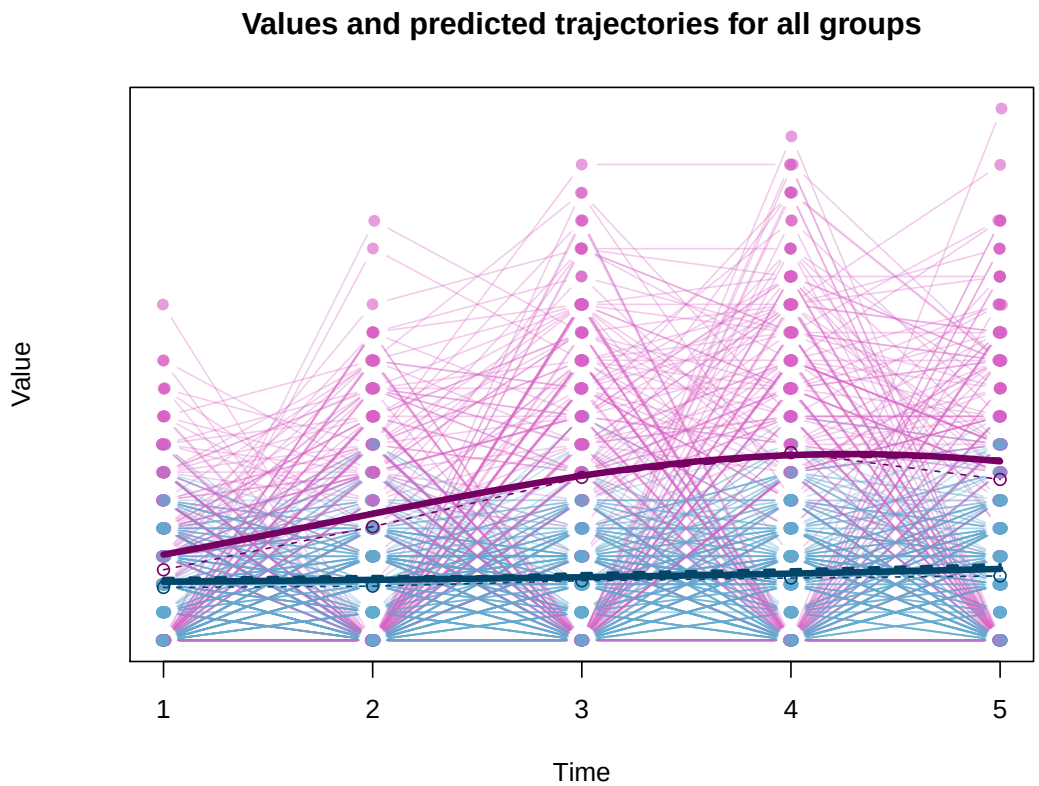
We can use all method too.

```
solEMTCOV = trajeR(Y = data[,2:6], A = data[,7:11], TCOV = data[,13:17],
                   ng = 2, degre=c(2,2), degre.nu = c(1,1),
                   Model="ZIP", Method = "EM", hessian = TRUE)
solEMIRLSTCOV = trajeR(Y = data[,2:6], A = data[,7:11], TCOV = data[,13:17],
                       ng = 2, degre=c(2,2), degre.nu = c(1,1),
                       Model="ZIP", Method = "EMIRLS", hessian = TRUE)
```

We have

SolLTCOV	SolEMTCOV	SoEMIRLSTCOV
Beta 1		
0.88080	0.88055	0.88074
0.01535	0.01543	0.01538
0.00677	0.00677	0.00677
Beta 2		
1.04296	1.04209	1.04295
0.61468	0.61497	0.61469
-0.07766	-0.07769	-0.07767
Nu 1		
-1.07073	-1.07096	-1.07079
0.00232	0.00237	0.00234
Nu 2		
0.02787	0.02778	0.02789
-0.16707	-0.16703	-0.16707
TCOV		
0.06373	0.06382	0.06375
0.01298	0.01319	0.01297
Pi		
0.65197	0.65195	0.65198
0.34803	0.34805	0.34802

We can add the effect of other time covariate.



8 Non Linear model

We suppose that the variable Y_{it} is defined by

$$y_{it} = f(a_{it}; \beta_k) + \epsilon_{it} \tag{10}$$

where $\epsilon_{it} \sim \mathcal{N}(0;\ \sigma_k)$.

In this case, the function f_k is non linear in the parameters β_k and

$$E(Y_{it} = y_{it} | W_i = w_i, C_i = k) = f(a_{it}; \beta_k) \tag{11}$$

We note ϕ the density function of a normal distribution with mean 0 and standard deviation 1 and Φ its cumulative distribution function, if we fit a group k we can write

$$P(Y_{it} = y_{it} | W_i = w_i, C_i = k) = \frac{1}{\sigma_k} \phi \left(\frac{y_{it} - f(a_{it}; \beta_k)}{\sigma_k} \right) \tag{12}$$

8.1 Parameters

Example
We use artificial data contained in the library. This are 500 longitudinal data generated by the

parameters above :

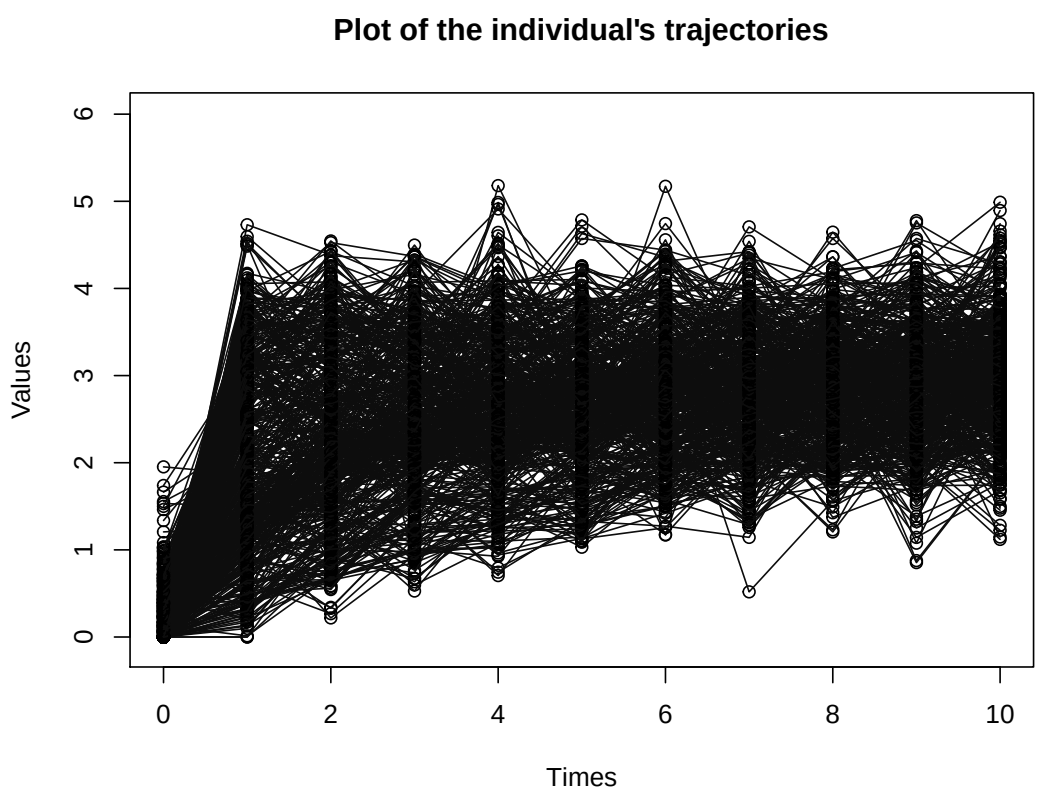
- 4 groups ;
- probability is $\pi_1 = 0.2$, $\pi_2 = 0.3$, $\pi_3 = 0.14$ and $\pi_4 = 0.36$;
- period is 11 ;
- $\sigma_1 = 0.45$, $\sigma_2 = 0.65$, $\sigma_3 = 0.35$ and $\sigma_4 = 0.4$.
- the link[1] function between individuals and parameters is

$$f(t; \beta_k) = \frac{\beta_{k1} t}{\beta_{k2} + t}$$

- the parameters[2] are $\beta_1 = (3.6, 0.025)$, $\beta_2 = (2.9, 0.06)$, $\beta_3 = (3.4, 5)$ and $\beta_4 = (3.4, 1.5)$.

The data contains the time variable dependent Y_i in `data[,2:12]`, the time variable in `data[,13:23]`.

- `data[,2:12]` is a matrix with real.
- `data[,13:23]` is a matrix with time 0 to 10.



[1]it is the Michaelis–Menten equation.

[2]the informed reader will have recognized, in order, *Glutamate dehydrogenase* in NAD^+, *Pyruvate carboxylaste* in ATP, *Threonine deaminase* in Threonine and *Hexokinase* in Fructose.

We use the non linear method to fit the model. We specify the number of group `ng` to 4 and the number of parameters `nbvar` to 2. We specify too `hessian=TRUE` to ask the calculus of the hessian matrix.
`itermax` is set to 300 to assure a good approximation of the parameters.

We call `trajeR` with option `Method ="EM"` and `Model="NL"`. By analyzing the graph above, we fit initial parameters as

```
paraminit=c(0.25,0.25,0.25,0.25,2,0.1,2.4,0.1,2.8,0.1,3,0.1,0.2,0.2,0.2,0.2)
```

Then we have to define the link function and its differential.

```
#defintion of the function
fct <- function(t, betak, TCOV){
  return(
    (betak[1]*t)/(betak[2]+t)
  )
}
#defintion of the differential
diffct <- function(t, betak, TCOV){
  return(c(
    t/(betak[2]+t),
    -(betak[1]*t)/(betak[2]+t)**2
  ))
}
```

We use the `trajeR` function with the parameters above

```
solEM = trajeR(Y = data[,2:12], A = data[,13:23],
               ng = 4, nbvar = 2,
               Method = "EM", Model = "NL", hessian = TRUE,
               fct = fct, diffct = diffct, paraminit = paraminit)
```

```
solEM

## Call TrajeR with 4 groups and a 1,1,1,1 degrees of polynomial shape of trajectory.
## Model : Non Linear
## Method : Expectation-maximization
##
##   group   Parameter   Estimate   Std. Error   T for H0:   Prob>|T|
##                                                 param.=0
## ---------------------------------------------------------------------
##       1   Intercept    3.30542      0.18506    17.86117          0
##              Linear    4.71859      0.60508     7.79825          0
##
##       2   Intercept    3.17397      0.06338    50.07605          0
##              Linear    1.29635      0.12496    10.37439          0
##
```

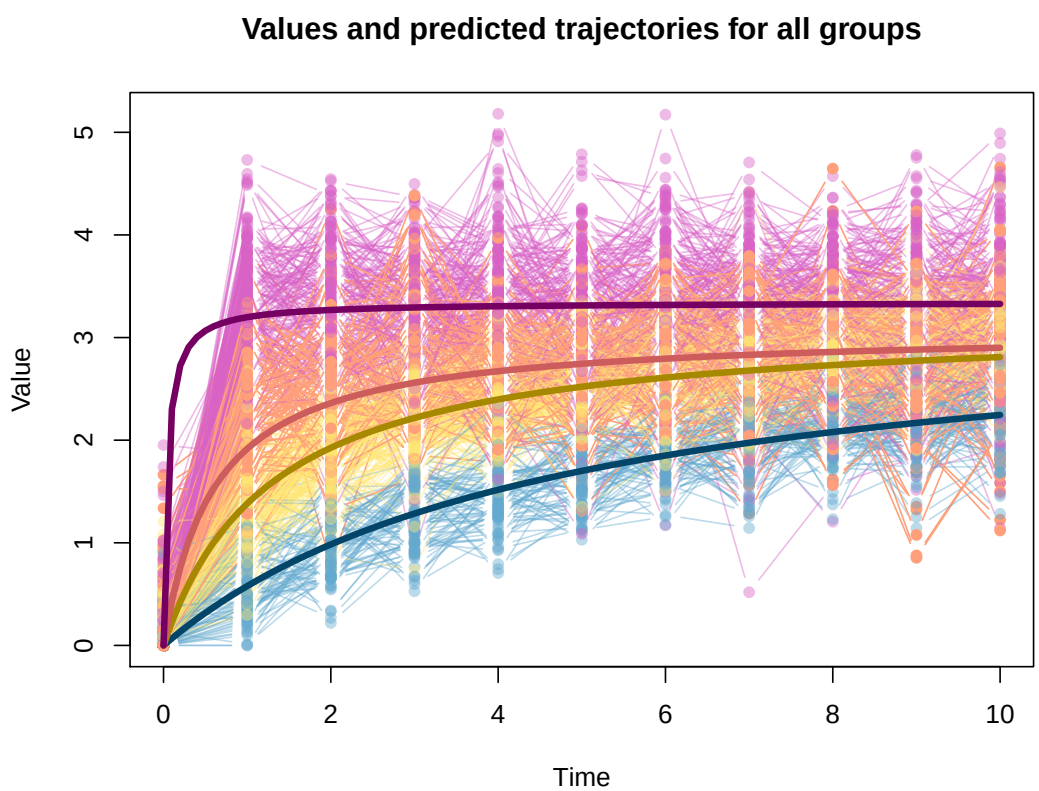
```
##       3   Intercept    3.07645      0.06425    47.88329          0
##              Linear    0.60537      0.11135     5.43645          0
##
##       4   Intercept    3.34255      0.02515   132.91026          0
##              Linear    0.04503       0.0236     1.90772    0.05648
## ---------------------------------------------------------------------
##       1      sigma1    0.63465          NaN         NaN        NaN
##       2      sigma2    0.56821       0.0036   157.93961          0
##       3      sigma3    0.60047      0.02216     27.0945          0
##       4      sigma4    0.71747      0.01667     43.0461          0
## ---------------------------------------------------------------------
##       1         pi1    0.15427      0.01646      9.3702          0
##       2         pi2    0.23712      0.02962     8.00592          0
##       3         pi3    0.25955          NaN         NaN        NaN
##       4         pi4    0.34906          NaN         NaN        NaN
## ---------------------------------------------------------------------
## Likelihood : -4891.536
```

If we compare this results with the theoretical values we have

Theoritical	Estimate
Beta 1	
3.400	3.30542
5.000	4.71859
Beta 2	
3.400	3.17397
1.500	1.29635
Beta 3	
2.900	3.07645
0.060	0.60537
Beta 4	
3.600	3.34255
0.025	0.04503
Sigma	
0.350	0.63465
0.400	0.56821
0.650	0.60047
0.450	0.71747
Pi	
0.140	0.15427
0.360	0.23712
0.300	0.25955
0.200	0.34906

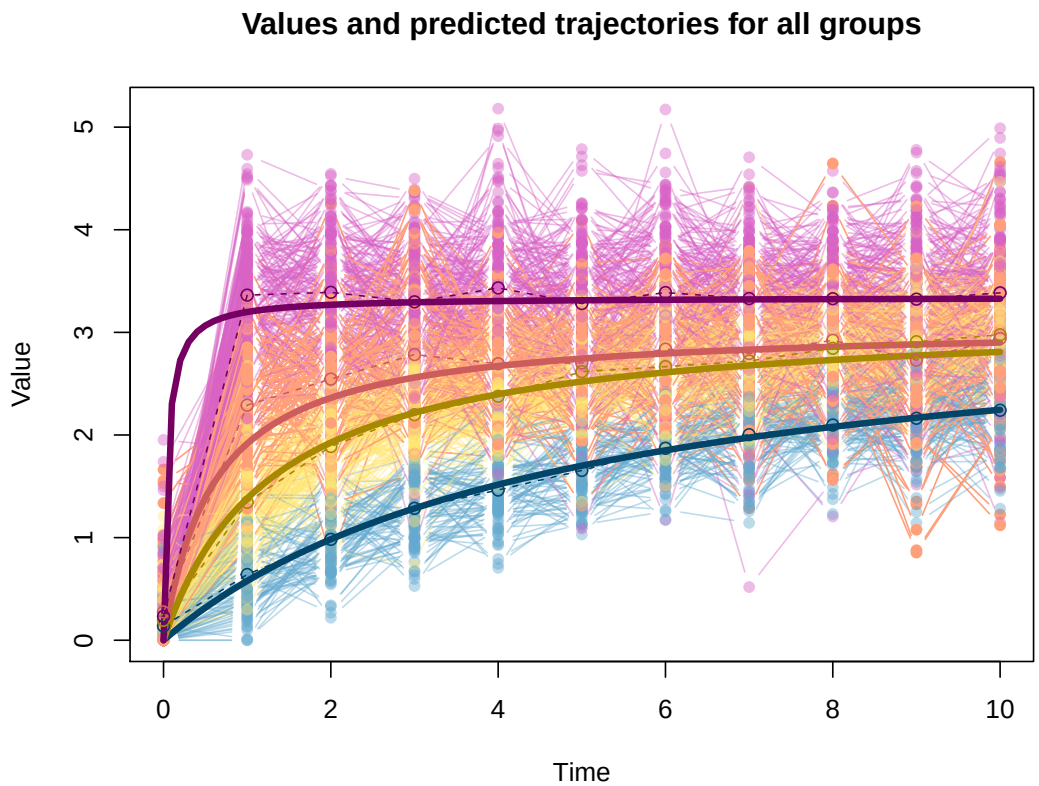
We plot the graph of the results.

```
trans = "70"
col1 = "#034569"
col1.1 = paste0("#64AAD0", trans)
col2 = "#750062"
col2.1 = paste0("#D962C7", trans)
col3 = "#A68900"
col3.1 = paste0("#FFE773", trans)
col4 = "#CD5C5C"
col4.1 = "#FFA07A"
cols1 = c(col1.1, col3.1, col4.1, col2.1)
cols2 = c(col1, col3, col4, col2)
vcol = c(cols1, cols2)
plot(solEM, Y=data[,2:12], A=data[,13:23],col = vcol)
```



Here too we can add mean of each epoch to the plot.

```
plot(solEM, Y=data[,2:12], A=data[,13:23],col = vcol, mean = TRUE)
```



9 Membership probability

Sometimes it is util to find the membership probability of an individual for each group. The function `GroupProb` do it. For each individual in the data it calculate the probability of membership of each group. The result is a matrix of real.

To illustrate this point we use the data and the results in the part page 21.

```
prob = GroupProb(solLs, Y =  data[,2:11], A = data[,12:21])
head(prob, n = 8)

##                Gr1          Gr2          Gr3
## [1,] 1.000000e+00 1.799408e-08 1.781027e-15
## [2,] 1.930888e-07 2.274434e-16 9.999998e-01
## [3,] 1.000000e+00 2.204867e-09 2.587772e-15
## [4,] 1.000000e+00 2.868289e-08 6.051662e-16
## [5,] 1.825766e-11 2.046285e-20 1.000000e+00
## [6,] 9.999999e-01 1.116069e-07 2.165143e-13
## [7,] 9.999998e-01 1.938767e-07 9.614756e-16
## [8,] 9.988100e-01 1.190018e-03 6.047510e-11
```

For example, for the individual 8, the probability to belong to the group 1 is almost 0.9988, to the group 2 is almost 0.0012 and to the group 3 is almost 0.

10 Profiles of group

One interesting aspect to adding covariate membership probability is the capacity to construct group profile by doing a cross tabulation of individual level trajectory group assignments with individual level characteristic that might be associated with trajectory group membership.
The function `GroupProfiles` construct this table of profile. If the covariate are continuous, the values in the table correspond to the mean of each variable else they can be interpreted as a percentage.

```
GroupProfiles(solLRisk, Y =  data[,2:11], A = data[,12:21], X = data[, 42:43])

##          Gr 1      Gr 2      Gr 3
## V42 0.4924891 0.5075429 0.5009375
## V43 0.4700437 0.5187429 0.5144792
```

Here the variable V42 and V43 are continuous, thus the values in this table are the mean of this two variables for each group.

11 Information criteria

To choice between several model, we can use information criteria. We have implement 3 classical methods, BIC, AIC and the SH (Slope Heuristic).

```
trajeRBIC(solLRisk)

## [1] 29255.68

trajeRAIC(solLRisk)

## [1] 29167.18
```

`trajeRSH` need a list with a least 10 objects of type trajectory to work.

12 Model adequacy

We follows the different tools proposed in his book by Nagin (2005).

AvePP

We call Average Posterior Probability (AvePP) the average posterior probability of membership for each group for those individuals that were assigned to. In a ideal situation the assignment probability for each individual would be 1 and the Average Posterior Probability (AvePP) would be 1 too.

```r
AvePP(solLRisk, Y = data[,2:11], A = data[,12:21], X = data[, 42:43])

## [1] 0.9957070 0.9920117 0.9999998
```

We can see that all values are near 1. It means that the model fit correctly the data.

Odds of Correct Classification

The Odds of Correct Classification for group k ($\mathrm{OCC_j}$) is the ratio between the odds of a correct classification into group j on the basis of the posterior probability rule and the odds of correct assignment based on random assignments with the probability of assignment to group j is done with $\hat{\pi_k}$, the probability estimate by the model. Larger value of OCC_k indicate better assignment accuracy

```r
OCC(solL, Y = data[,2:11], A = data[,12:21])

## [1] 2.769705e+02 2.269346e+02 2.524712e+07
```

We have big values. The model the model fit correctly the data.

Estimated group probabilities versus proportion of the sample assigned to the group

We calculate the probability of group membership by two ways : using $\hat{\pi_k}$ or using the proportion P_k of the sample assigned to the group k. Ideally the this two values would be equal.

```r
propAssign(solL, Y =data[,2:11], A = data[,12:21])

## gr
##     1     2     3
## 0.458 0.350 0.192
```

We can compare with $\hat{\pi_k}$ ca calculate by the model.

```r
exp(solL$theta)/sum(exp(solL$theta))

## [1] 0.4589082 0.3489514 0.1921404
```

This values are close, thus the model the model fit correctly the data.

Confidence interval

A narrow confidence interval of $\hat{\pi_k}$ implies that the probability is accurately estimated. This intervals are calculated with bootstrap method.

```r
ConfIntT(solL, Y = data[,2:11], A = data[,12:21], nb = 10000, alpha = 0.98)

##           [,1]      [,2]      [,3]
## 1%  0.4133988 0.3074413 0.1614733
## 99% 0.5045751 0.3928050 0.2269818
```

Adequacy

We can summarize the method above in one table.

```
adequacy(solL,  Y = data[,2:11], A = data[,12:21], nb = 10000, alpha = 0.98)

##                       1           2            3
## Prob. est.   0.4589082   0.3489514 1.921404e-01
## CI inf.      0.4137369   0.3074233 1.623457e-01
## CI sup.      0.5054918   0.3928927 2.251117e-01
## Prop.        0.4580000   0.3500000 1.920000e-01
## AvePP        0.9957610   0.9918456 9.999998e-01
## OCC        276.9705045 226.9346306 2.524712e+07
```